

JOI 2024/2025 本選 1 問目 色塗り (Grid Coloring) 解説

解説担当：西脇響喜

問題概要

- $N \times N$ のマス目に色を塗る (色は正整数の番号で表される).
- 1 行目のマスと 1 列目のマスの色は既に決まっていて、
残りのマスを (i, j) ($2 \leq i, j \leq N$) の昇順に
 - i. マス $(i - 1, j)$ に塗られている色と
 - ii. マス $(i, j - 1)$ に塗られている色のうち
番号の大きい方の色で塗る.
- 最終的に最も多くのマスに塗られた色および、その色が塗られているマスの数を求めよ.

$$N \leq 2 \times 10^5, \text{ (色の番号)} \leq 10^9$$

小課題

番号	配点	制約	計算量の例
1.	15 点	$N \leq 500, C \leq 10^5$	$O(N^2 + \max(C))$
2.	10 点	$N \leq 500$	$O(N^2 \log N)$
3.	20 点	$C \leq 2$	$O(N)$
4.	25 点	A_i, B_j は狭義単調増加	$O(N)$
5.	30 点	追加の制約はなし	$O(N \log N)$

(色の番号の最大値 ($\equiv$ 値域) を C で表した.)

小課題 1 ($N \leq 500, C \leq 10^5$)

- N が小さい
→ N^2 個のマスの塗られる色の番号を愚直に求めても間に合う！

小課題 1 ($N \leq 500, C \leq 10^5$)

- N が小さい
→ N^2 個のマスの塗られる色の番号を愚直に求めても間に合う！
- $O(N^2)$ 回の計算でマスの色を求めた後、各色の出現数を $O(C)$ で求める
- 長さ C の配列を用意した上で、 N^2 個の各マスについて塗られている色に対応する要素に 1 を足していく
(色の番号が 110 なら 110 番目の要素に 1 を足す、といった具合)

小課題 2 ($N \leq 500$)

- C が大きい
→ 長さ C の配列を用いることができない

小課題 2 ($N \leq 500$)

- C が大きい
 - 長さ C の配列を用いることができない
 - 代わりに map (連想配列) を使う
- A_i, B_j に現れない番号の色は無視して良いので、考えるべき色の種類数は高々 $O(N)$
- 計算量は $O(N^2 \log N)$

小課題 3 ($C \leq 2$)

- N が大きい
 - 愚直に N^2 個のマスの色を求めている間は間に合わない
- しかし C が非常に小さい (色の番号が高々 2 通り)
 - どんなときにマス (i, j) が 2 で塗られるか？を考える

小課題 3 ($C \leq 2$)

こんな例を考える

2	1	1	2
1			
2			
1			

小課題 3 ($C \leq 2$)

2	1	1	2
1	1	1	2
2			
1			

小課題 3 ($C \leq 2$)

2	1	1	2
1	1	1	2
2	2	2	2
1			

小課題 3 ($C \leq 2$)

2	1	1	2
1	1	1	2
2	2	2	2
1	2	2	2

小課題 3 ($C \leq 2$)

考察

- 2 が現れて以降の行はすべて 2

小課題 3 ($C \leq 2$)

考察

- 2 が現れて以降の行はすべて 2
→ 同じことが列についても実は言える
→ 2 が現れて以降の列はすべて 2
- 反対に、2 が現れるまではずっと 1
- まとめると、マス (i, j) ($2 \leq i, j \leq N$) は
 - i. $A_2 = A_3 = \dots = A_i = B_2 = B_3 = \dots = B_j = 1$ ならば 1
で塗られる
 - ii. そうでないならば 2 で塗られる

小課題 3 ($C \leq 2$)

数えるパート

- $A_2 = A_3 = \dots = A_{i'} = 1$ が成り立つ最大の i を i' ,
 $B_2 = B_3 = \dots = B_{j'} = 1$ が成り立つ最大の j を j' として,
($A_2 = 2$ なら $i' = 1$, $B_2 = 2$ なら $j' = 1$)
2 行目、2 列目以降の $(N - 1)^2$ マスのうち
 - i. 1 で塗られるものの個数は $(i' - 1) \times (j' - 1)$
 - ii. 2 で塗られるものの個数は $(N - 1)^2 - (i' - 1) \times (j' - 1)$
- 1 行目, 1 列目のマスについても個数に足すのを忘れないように！

小課題 3 ($C \leq 2$)

- 計算量は $O(N)$
- 各色について「その色で塗られるマス of 個数はいくつ?」を数えるのはかなり本質的な考察
(満点制約においても色の候補数は高々 $O(N)$ なので, 数えるパートを高速に行うことで満点が得られる)

小課題 4 (A_i, B_j は狭義単調増加)

こんな例を考える

1	3	5	8
2	3	5	8
4	4	5	8
8	8	8	8

小課題 4 (A_i, B_j は狭義単調増加)

考察

- マス (i, j) ($2 \leq i, j \leq N$) は $\max(A_i, B_j)$ で塗られている
→ 理由:
 - $A_i \leq B_j$ として
 - a. マス $(2, j), (3, j), \dots, (i-1, j)$ は B_j で塗られ
 - b. マス $(i, j-1)$ の色は B_j 以下となる ($A_2, A_3, \dots, A_i, B_2, B_3, \dots, B_{j-1} \leq B_j$ に依る) ため
 - 細かい証明は数学的帰納法で

小課題 4 (A_i, B_j は狭義単調増加)

数えるパート

- 各行, 各列に注目！！
- 例えば各行 i ($2 \leq i \leq N$) について,
「 i 行目の 2 列目以降にあるマスのうち A_i で塗られるものの個数はいくつか？」を考える
→ これは $B_j \leq A_i$ なる j ($2 \leq j \leq N$) の個数に等しい
→ B には単調性があるので, 二分探索をすると $O(\log N)$ で求まる
- 列についても同じことをやる
- $A_i = B_j$ なる (i, j) が存在する場合に注意！
(あとで重複分を引いておくなどの処理が必要)

小課題 4 (A_i, B_j は狭義単調増加)

- 小課題 2 の map 方針と組み合わせるなどして, $O(N \log N)$

小課題 5 (追加の制約なし)

考察

- 小課題 4 について: なんで $\max(A_i, B_j)$?

小課題 5 (追加の制約なし)

考察

- 小課題 4 について: なんで $\max(A_i, B_j)$?
→ $\max(A_2, A_3, \dots, A_i) = A_i$, $\max(B_2, B_3, \dots, B_j) = B_j$ だから
- より一般に, マス (i, j) ($2 \leq i, j \leq N$) は
 $\max(A_2, A_3, \dots, A_i, B_2, B_3, \dots, B_j)$ で塗られる
→ 証明は帰納法で

小課題 5 追加の制約なし

6	4	2	6
3	4	4	6
5	5	5	6
1	5	5	6

小課題 5 追加の制約なし

数え上げパート

- $A'_i = \max(A_2, A_3, \dots, A_i)$, $B'_j = \max(B_2, B_3, \dots, B_j)$ として,
 A' と B' について小課題 4 と同じ方針で数えれば OK
- 重複分を除くパートが少し面倒かも...?
→ 色 X についての重複分は $(A'_i = X \text{ なる } i \text{ の個数}) \times (B'_j = X \text{ なる } j \text{ の個数})$ で求まり, 実は場合分けも要らない

小課題 5 (追加の制約なし)

- より簡明な実装例が後ほど JOI のホームページに上がるので、参考にしてみてください
(「 $A'_i = B'_j$ の場合 $A'_i < B'_j$ と同様に扱う」といった無理やりな順序付けをしても実は壊れない)
(こういう無理やりな順序付けは大抵の場合壊れず、場合分けも省いてくれるので、一般にとっても有用)
- 計算量は $O(N \log N)$

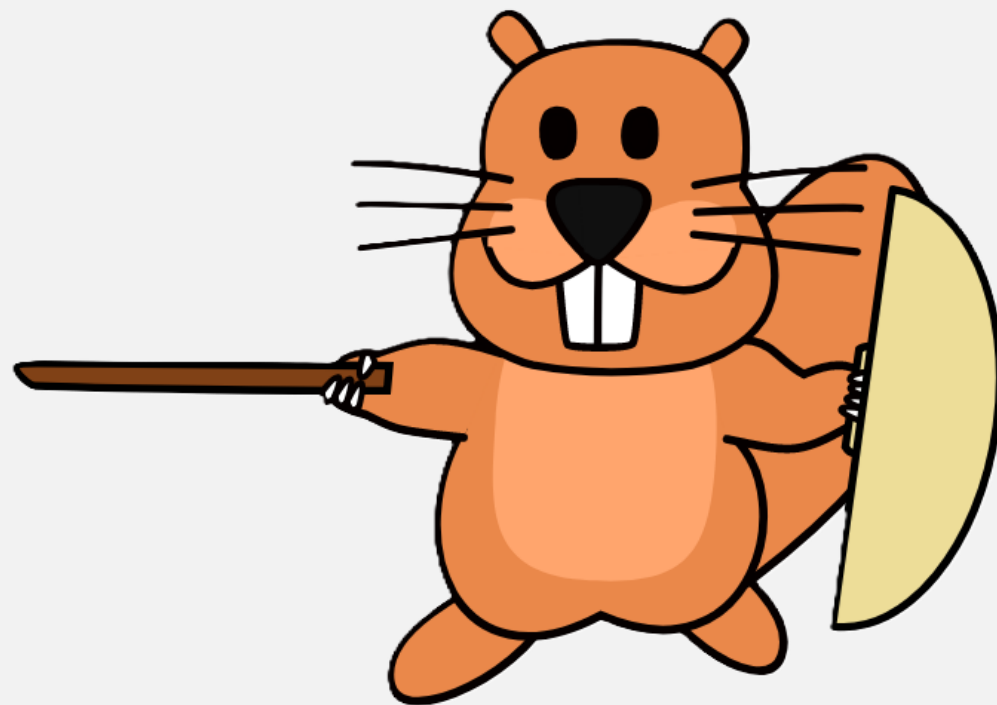
統計情報

点数	1	2	3	4	5	人数	累積人数
100	○	○	○	○	○	164	191
70	○	○	○	○	-	2	27
50	○	○	-	○	-	3	25
45	○	○	○	-	-	5	22
25	○	○	-	-	-	4	17
15	○	-	-	-	-	2	13
0	-	-	-	-	-	11	11

JOI2024/2025 本選 2 問目

勇者ビ太郎 2 (Bitaro the Brave 2) 解説

解説: ヘフアナン色葉(iro_)



問題概要

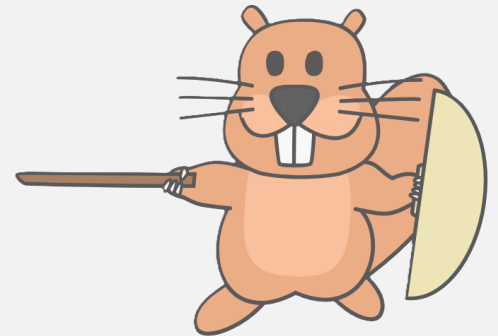
0 問題概要

x と i を決める

$i, i + 1, i + 2, \dots, N, 1, \dots, i - 1$ の順番に下の条件を満たすか確認する

- $x \geq A_i$ であるか判定し、 x に B_i 加算する

すべての地点において条件を満たす x の最小値を求める

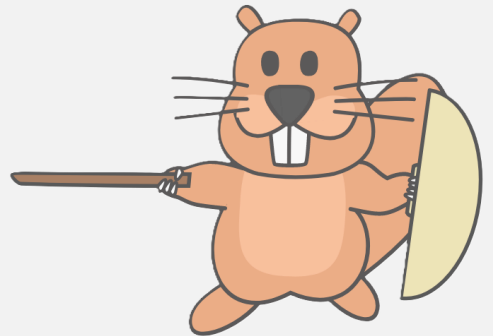


0 制約

$$2 \leq N \leq 500\,000$$

$$0 \leq A_i \leq 10^9 \quad (1 \leq i \leq N)$$

$$0 \leq B_i \leq 10^9 \quad (1 \leq i \leq N)$$



0 小課題

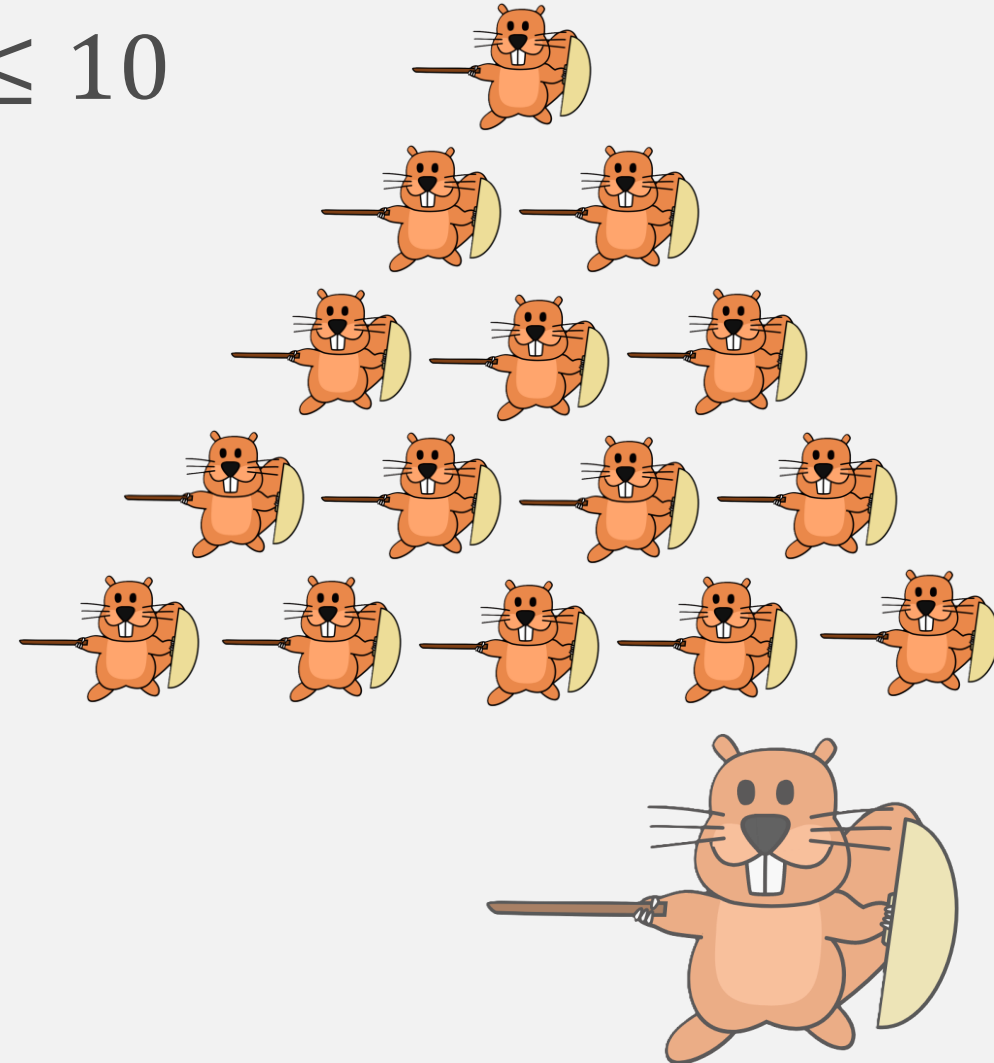
小課題1 10 点 : $N \leq 2\,000$, $x \leq 10$

小課題2 21 点 : $N \leq 2\,000$

小課題3 19 点 : $x \leq 10$

小課題4 22 点 : $B_i \leq 1$

小課題5 28 点 : 追加制約なし



その前に簡単な Tips

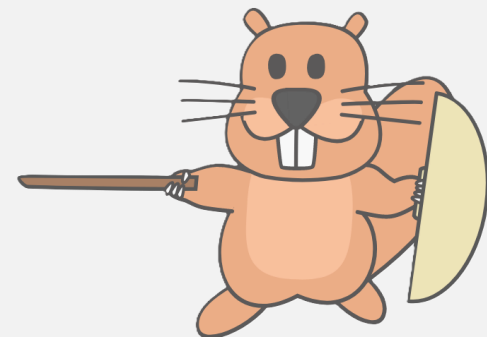
0 円環

円環になっている！

配列の長さを倍にして $i + N$ 番目に i 番目と同じ値

一番強いモンスターを倒せるようになったらよい

A の最大値が最後尾になるようローテーション



小課題 1

$$N \leq 2\,000, x \leq 10$$

1 初期値 x と開始地点 i を固定

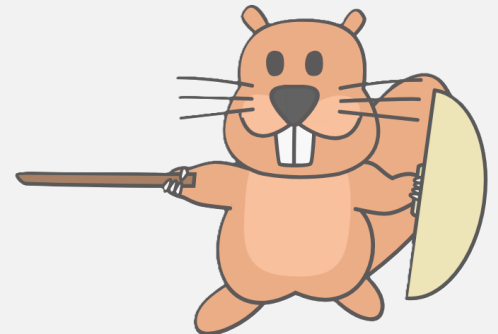
初期値 x 開始地点 i を全探索

$y(\text{現在の強さ}) = x, j = 0, 1, 2, \dots, N - 1$

- $y \geq A_{i+j}$ であるか判定 (true / false)
- $y = y + B_{i+j}$

$O(N)$ で判定

$O(ans \times N^2)$



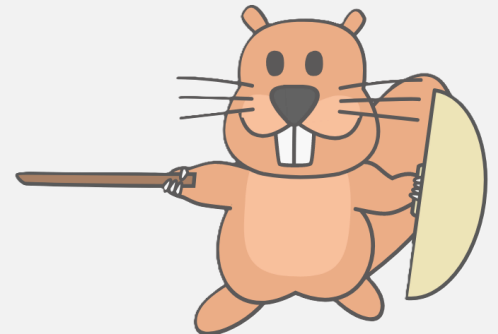
小課題 2

$$N \leq 2\,000$$

2 初期値 x

初期値 x は初めに決めなくてもよい
判定するときに足りない分足せばよい

→ 開始地点 i のみ固定



2 開始地点 i のみ固定

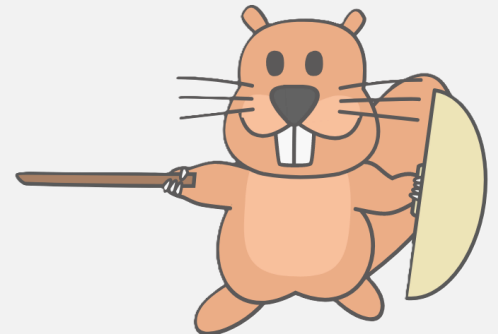
開始地点 i を全探索

$$y = x = 0, j = 0, 1, 2, \dots, N - 1$$

- $y < A_{i+j}$ のとき、 $x = x + (A_{i+j} - y)$, $y = A_{i+j}$
- $y = y + B_{i+j}$

$O(N)$ で判定

$O(N^2)$



小課題 3

$$x \leq 10$$

3 初期値 x のみ固定

初期値 x を全探索

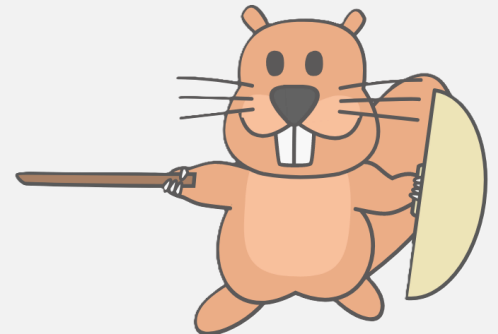
$$y = x, j = 0, 1, 2, \dots, 2 \times N - 1$$

N 箇所連続で条件を満たすところがあればよい

- $Y \geq A_j$ であるか (true / false)
- $y = y + B_j$

$O(N)$ で判定

$O(ans \times N)$



小課題 4

$$B_i = 1$$

4 条件を整理しよう

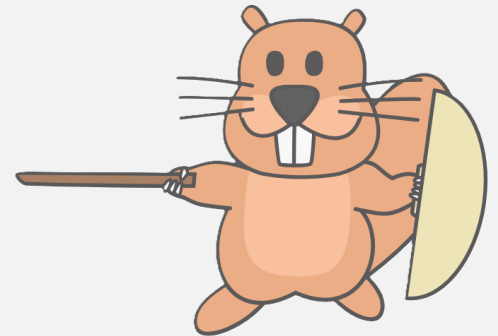
$B_i = 1$ なので

開始地点からの距離の分、ビ太郎の強さが増加する

開始地点 i でモンスター j を倒すときまでに強さは

- $j + N - i$ (if $j < i$)
- $j - i$ (if $j \geq i$)

増加する

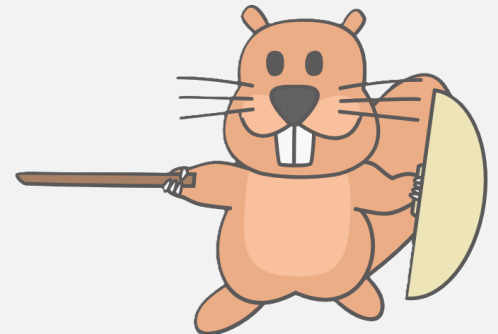


4 条件を整理しよう

i 番目のモンスターから始めたとき
各モンスター j を倒すのに必要な初期の強さ

- $A_j - (j + N - i)$ (if $j < i$)
- $A_j - (j - i)$ (if $j \geq i$)

これの最大値を求めればよい



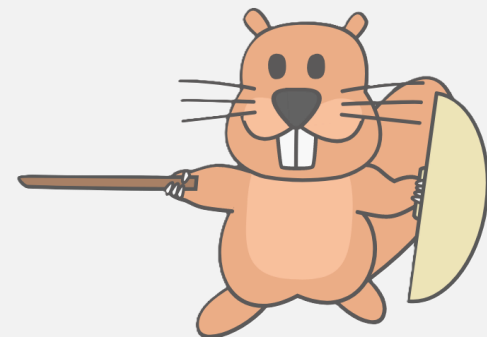
4 式を整理して

よって i を決めたとき

$$x = i + \max \begin{cases} A_j - (j + N), & j < i \\ A_j - j, & j \geq i \end{cases}$$

左右からの累積 max を求めれば高速に求まる

$O(N)$



小課題 5

追加制約なし

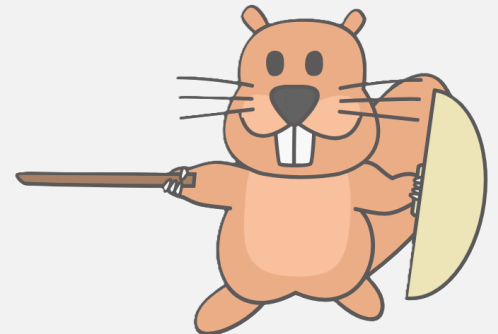
5 条件を整理しよう

$C_i = B_1 + B_2 + \dots + B_{i-1}$ とする

i 番目のモンスターから始めたとき
各モンスター j を倒すのに必要な初期の強さ

- $A_j - (C_j + C_{N+1} - C_i)$ (if $j < i$)
- $A_j - (C_j - C_i)$ (if $j \geq i$)

これの最大値を求めればよい



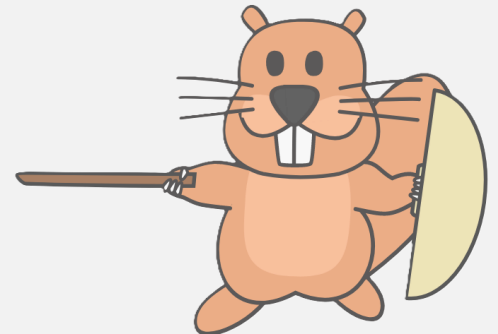
5 式を整理して

よって i を決めたとき

$$x = C_i + \max \begin{cases} A_j - (C_j + C_{N+1}), & j < i \\ A_j - C_j, & j \geq i \end{cases}$$

左右からの累積 max を求めれば高速に求まる
セグメント木を使ってもよい

$O(N)$

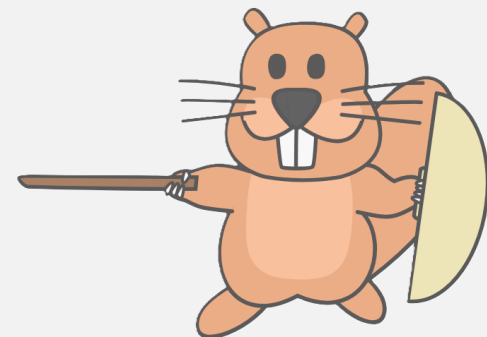


5 別解

小課題 3 より初期値 x を決めたら $O(N)$ で条件を満たすか判定できる

答えを二分探索しても解ける！

$$O(\log(\max A) \times N)$$



得点分布

100点: 154 人

72点: 3 人

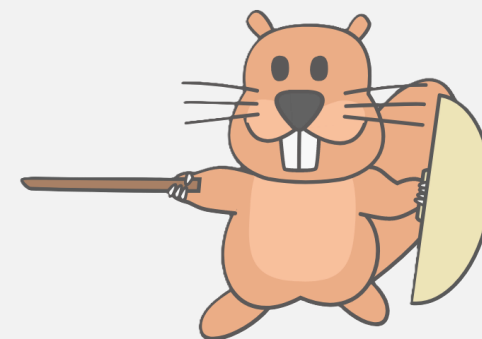
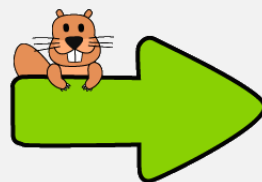
53点: 2 人

51点: 1 人

31点: 18 人

10点: 2 人

0点: 11 人





JOI 2024/2025 本選 3 問目 ミ・テレフェリコ (Mi Teleférico) 解説

解説: 蜂矢 倫久 (Mitsubachi)

Step 0

問題概要

0

問題概要

- ボリビアの首都ラパスにはミ・テレフェリコ (Mi Teleférico) というロープウェイが通っています
- 標高が 3700m 近くあり高低差のある移動は大変なので作られました
- 意味はスペイン語で「私のロープウェイ」

0

問題概要

- N 頂点 M 辺のグラフがあります
- 辺は有向辺で各辺には C_i という値がある
- 辺が A から B にあるとき、 $A < B$ が成立
- フリーパス (l, r) を用いると $l \leq C_i \leq r$ なる辺を使える
- 使える辺のみで頂点 1 から全部の頂点に行けるようにしたい
- そこで次の操作を行う

0

問題概要

- フリーパス (l, r) とフリーパス (l', r') を交換する
- この操作のコストは $|l - l'| + |r - r'|$
- Q 個のクエリに答える j 番目のクエリは以下のように書ける
- フリーパス (L_j, R_j) を持っている状態からコスト X_j 以内の交換をして目標を達成できるか判定してください

0

制約

- $N, M \leq 300\,000$
- $Q \leq 400\,000$
- $P, X_j \leq 10^9$
- $C_i, L_j, R_j \leq P$

小課題番号	N, M	P	Q	X_j	配点
1	50		50	0	7 点
2		10			8 点
3		100			11 点
4		300 000		0	23 点
5		300 000			9 点
6	8 000				22 点
7					20 点

Subtask 1

$$N, M, Q \leq 50, X_j = 0$$

1

$X_j = 0$ の意味

- $X_j = 0$ のとき可能な交換は $|l - l'| + |r - r'| = 0$ な場合だけ
- これは $l = l', r = r'$ なので異なるフリーパスと交換できない
- つまり「今あるフリーパスで全部いけるか判定してください」という形になる

1

グラフアルゴリズム

- フリーパスが決まっていれば使える辺の一覧も $O(M)$ で分かる
- 使える辺が分かれば BFS, DFS, Dijkstra, ワーシャルフロイドなどのアルゴリズムを用いてどの頂点に到達できるか分かる
- 計算量はクエリあたり BFS とかなら $O(N + M)$ で、ワーシャルフロイドなら $O(N^3 + M)$ で可能
- 全体として $O((N + M)Q)$ や $O((N^3 + M)Q)$ で解ける

1

発展的な話

- 時間に余裕がなければカットします
- ワーシャルフロイドでやってることは
- $d[i][j]$ を $i \rightarrow j$ の最短路として $d[i][j] \leftarrow \min(d[i][j], d[i][k] + d[k][j])$ として求めるというもの
- 正しいループ順: $k \rightarrow i \rightarrow j$
- 間違ったループ順: $i \rightarrow j \rightarrow k$ (などなど)

1

発展的な話

- 間違ったループ順でも 3 回やれば正しいことが知られてる
- 今回は $i \rightarrow j \rightarrow k$ で 1 回しかしなくても正当
- 証明: $i \rightarrow j$ の最短路のうち $k \rightarrow j$ 辺を使うものがあるとして $d[i][k] + d[k][j]$ を計算するとき $d[i][k]$ が正しく計算されていれば OK
- $j = i + 1, i + 2, \dots$ の順に帰納法を回すと OK
- よく見ると区間 DP の計算方法と同じ

Subtask 2

$$P \leq 10$$

2

候補について考える

- フリーパスとしてありうるのは $1 \leq l \leq r \leq P$ を満たすもののみ
- つまり $O(P^2)$ 種類しかない
- あるフリーパスが条件を満たすか → 小課題 1 で議論した通り BFS や DFS など OK
- 前計算として $O((N + M)P^2)$ かけて全フリーパスについて条件を満たすか計算
- ワーシャルフロイドだと間に合わない

2

クエリの処理

- クエリの処理を考える
- 交換先は $O(P^2)$ 種類 コストはそれぞれ $O(1)$ で計算できる
- つまり交換する候補についてコストを計算して、交換可能ならそれが条件を満たすか前計算から確認すれば OK
- 全体で $O((N + M + Q)P^2)$ で解けました

Subtask 3

$$P \leq 100$$

3

前計算について

- 小課題 2 と同様に前計算を考える
- 枝刈りできそうなパートを考える
- l を固定して r を動かすとき、OK になったら以降は見なくていい (r を増やして使えた辺が使えなくなることはないため)
- l を増やして r を再び動かすとき、 r のスタートは先ほど調べたときの r としてよい

3 具体例

- $\text{check}(l, r)$ を (l, r) について条件を満たすか調べる関数とする
- 具体例で書いてみる $P = 4$ として
- $l = 1$: $\text{check}(1, 1) \rightarrow \text{check}(1, 2) \rightarrow \text{check}(1, 3)$
- $l = 2$: $\text{check}(2, 3) \rightarrow \text{check}(2, 4)$
- $l = 3$: $\text{check}(3, 4)$
- $l = 4$: $\text{check}(4, 4)$
- という感じ

3

尺取り法

- 定数倍しか改善してなさそうだが実は check を呼び出す回数は $O(P^2)$ から $O(P)$ へ改善している
- いわゆる**尺取り法**というアイデア
- 証明: $l + r$ の総和を見ると、1 ずつ増えていく これと $l, r \leq P$ より示される
- これを用いれば $O((N + M)P)$ で l を固定したときの r として必要な最小の値が分かる
- r として必要な最小の値を $f(l)$ とでも表記する

3

クエリの処理

- 各クエリでは交換するフリーパスの l を固定して考える
- 交換後の r について $f(l) \leq r$ となるようにしたい
- このために必要なコストは $\max(0, f(l) - R_j)$
- つまり $|L_j - l| + \max(0, f(l) - R_j) \leq X_j$ なる l が存在するか探す
- これは l あたり $O(1)$ でできるのでクエリあたり $O(P)$
- よって $O((N + M + Q)P)$ で解くことができました

Subtask 4

$P \leq 300\,000, X_j = 0$

4

条件の言い換え

- 今のところ使っていない問題の性質は $A_i < B_i$
- つまりグラフは DAG だということ
- DAG は有向辺からなる閉路がないグラフ
- これを使って条件が言い換えられるか考える
- とりあえず具体例で考えてみよう

4

条件の言い換え

- 頂点 2 に行きたいなら $1 \rightarrow 2$ の辺はほしい
- 頂点 3 に行きたいなら $1 \rightarrow 3$ か $2 \rightarrow 3$ はほしい
- 頂点 4 に行きたいなら $1 \rightarrow 4$ か $2 \rightarrow 4$ か $3 \rightarrow 4$ はほしい
- ...
- とりあえず頂点 1 以外の各頂点について、その頂点に入る辺がほしい
- 実はこれが必要十分条件になっている
- 必要条件を列挙すると十分条件になることはよくある

4

証明

- 必要性: ある頂点 v について $u \rightarrow v$ という形の辺がないなら頂点 v にはどうやってもいけない
- 十分性: どの頂点からも辺を逆にたどって頂点 1 にいければ OK
- どの頂点からも辺は入ってきている この辺を逆にたどると番号の小さい頂点にいける
- これを繰り返せばいずれは頂点 1 にたどりつくので OK

4

証明

- 必要性: ある頂点 v について $u \rightarrow v$ という形の辺がないなら頂点 v にはどうやってもいけない

つまり、辺の情報として重要なのは B_i のみ

- どの頂点からも辺は入ってきている この辺を逆にたどると番号の小さい頂点にいく

A_i を保存する必要はない！！

- これを繰り返せばいずれは頂点 1 にたどりつくので OK

4

$f(l)$ を求める

- 言い換えた条件をもとに $f(l)$ を求めよう
- 条件は $l \leq C_i \leq r$ かつ $B_i = x$ である i が x によらずあること
- $B_i = x$ である i における C_i において l 以上の中で最小のものを
使いたい
- 逆にこれを使えれば OK

4

具体例

- $N = 3$ において
- $B_i = 2$ である i における C の列が $(3, 7, 9)$ で
- $B_i = 3$ である i における C の列が $(4, 6, 9)$ とする
- $l = 5$ なら $(3, 7, 9)$ からは 7 を、 $(4, 6, 9)$ からは 6 を使いたい
- これらの $\max$ である 7 が $f(5)$
- $l = 10$ なら $(3, 7, 9)$ からも $(4, 6, 9)$ からも取れない
- こういうときは $f(10) = \text{inf}$ とでもしておこう
- 実装上は 10^{18} とか大きい数にしておけば OK

4

$f(l)$ を求める

- $f(l)$ を求める方法を考える
- B ごとに C を集めて sort した列を考える
- ある B について $(V_1, V_2, \dots, V_n)$ とあるとき
- $i = 1, 2, \dots, V_1$ について $f(i) \leftarrow \max(f(i), V_1)$ と更新
- $i = V_1 + 1, V_1 + 2, \dots, V_2$ について $f(i) \leftarrow \max(f(i), V_2)$ と更新
- ...
- $i = V_n + 1, V_n + 2, \dots, P$ について $f(i) \leftarrow \max(f(i), \text{inf})$ と更新
- ということをしたい

4

$f(i)$ を求める

- さっきの $\max$ が $+$ の形ならいわゆる imos 法をすれば OK
- 累積 $\max$ を使うと今回も同様にできる
- $f(1) \leftarrow \max(f(1), V_1), f(V_1 + 1) \leftarrow \max(f(V_1 + 1), V_2), \dots,$
 $f(V_n + 1) \leftarrow \max(f(V_1 + 1), \text{inf})$
- とあらかじめしておく 他の B についても同様
- 最後に $i = 2, 3, \dots, P$ の順で $f(i) \leftarrow \max(f(i), f(i - 1))$ とすれば OK
- 最後に $i \leq f(i)$ を反映しておく

4

$f(i)$ を求める

- さっきの具体例なら
- $B_i = 2$ である i における C の列が $(3, 7, 9)$ で
- $B_i = 3$ である i における C の列が $(4, 6, 9)$
- $f(1) \leftarrow \max(f(1), 3), f(4) \leftarrow \max(f(4), 7), f(8) \leftarrow \max(f(8), 9), f(10) \leftarrow \max(f(10), \text{inf})$
- $f(1) \leftarrow \max(f(1), 4), f(5) \leftarrow \max(f(5), 6), f(7) \leftarrow \max(f(7), 9), f(10) \leftarrow \max(f(10), \text{inf})$
- とあらかじめしておく

4

f(l) を求める

- 今の f の列は (4, -inf, -inf, 7, 6, -inf, 9, 9, inf, inf, ...) という形
- 累積 max を取ると (4, 4, 4, 7, 7, 7, 9, 9, inf, inf, ...) になる
- B ごとに C を集めて sort した列を作るのは全体で $O(M \log M)$
- あらかじめ max を取るのは $O(M)$
- 累積 max を取るのは $O(P)$
- f(l) を求める部分は $O(M \log M + P)$ でできた

4

別の方法

- 累積 max を取らなくてもいける
- l が小さいものから求めるとして各列の中で l 以上の最小のものがどれかを管理しておく これらのうち最大の値が $f(l)$
- ある列の中で l 以上の最小のものが l ならその列の見るべき index を 1 増やす 同時に最大の値も更新しておく
- そのあと $l \leftarrow l + 1$ とする
- index を増やす回数は $O(M)$ 回なので前と同様 $O(M \log M + P)$
- (B, C) が同じ組がある場合の実装に注意

4

クエリ の処理

- ここまで来ればクエリでは $f(L_j) \leq R_j$ かを判定するだけ
- 全体で $O(M \log M + P + Q)$ で解けました

4

さらに別解

- 条件を観察すると、条件を満たさないときは B が同じ C を集めた列のうちどれかで $V_i < l \leq r < V_{i+1}$ を満たしている
- つまり $[V_i + 1, V_{i+1} - 1]$ という形をした区間がたくさんあって各クエリでは区間 $[l, r]$ が与えられるのでたくさんある区間のどれかに包含されてないか判定してくださいというものになる
- どれかに包含されてるとき $V_i + 1 \leq l \leq r \leq V_{i+1} - 1$ が成立
- つまり区間の左端が l 以下なら、区間の右端は r 未満なら OK

4

さらに別解

- 「**区間**の左端が l 以下なら、区間の右端は r 未満」を判定したい
- あらかじめ **区間** を左端で sort しておく これは $O(M \log M)$
- すると判定したいものは sort した後に x 番目までに出てくる **区間** の右端の最大値が r 未満かというもの
- この x は lower_bound など求められる
- 累積 max をしておけば x が分かれば簡単
- 全体で $O((M + Q) \log M)$ で解けました

Subtask 5

$P \leq 300\,000$

5

$f(l)$ を求める

- $f(l)$ を求めるのは小課題 4 の方法でできる
- これまでと同じように交換するフリーパスの l を固定すると $O(PQ)$ になって間に合わない
- 交換する方法について考えてみる
- まず l を増やしたり r を減らすのは得ないので l を減らして r を増やす交換だけすればいい

5

交換について

- つまり $(l, r) \rightarrow (l - dl, r + dr)$ に変えると思う
- ここで $0 \leq dl, dr$ とする
- $f(l - dl) \leq r + dr$ としたい
- $dl + dr$ を小さくしたい
- 場合分けをする

5

交換について

- ① $f(l - dl) \leq r$ のとき
 - $dr = 0$ として OK
 - さらに、 dl はそのような中で最小のものを取れば良い
 - f は単調増加なので一度 $f(l - dl) \leq r$ になったら dl を増やしても同じ
- ② $f(l - dl) > r$ のとき
 - $dr = f(l - dl) - r$ とすれば OK

5

クエリ処理 ①

- ① $f(l - dl) \leq r$ のとき
- このような最小の dl を見つけるにはどうすればいいか
- 言い換えると $f(i) \leq r$ なる最大の i を見つける問題になる
- これは lower_bound など配列上の二分探索を使えば OK
- よって ① にするパターンは $O(\log P)$ で解けた

5

クエリの実処理 ②

- ② $f(l - dl) > r$ のとき
- ① の考察を用いてこのような dl の範囲は分かる
- つまりある区間の dl における $dl + dr = dl + f(l - dl) - r$ の最小値を知りたい
- 式変形をしていい形にしたい
- $dl = -(-dl)$ と思うと $f(l - dl) - (-dl) - r$ と変形できる
- $-dl = (l - dl) - l$ と思うと $f(l - dl) - (l - dl) + l - r$ と変形できる

5

クエリの処理 ②

- $f(l - d) - (l - d) + l - r$ の最小値を求める問題になった
- $g(x) = f(x) - x$ と思うと $g(x) + l - r$ の最小値を求める問題に
- l, r は入力における L_j, R_j に対応しているのである区間における $g(x)$ の最小値を求めればいい
- g を求めるのは簡単
- 区間における最小値を求めるといえば → **Segment Tree** や **Sparse Table** が使える

5

クエリ の処理 ②

- Segment Tree なら構築 $O(P)$ でクエリあたり $O(\log P)$
- Sparse Table なら構築 $O(P \log P)$ でクエリあたり $O(1)$
- クエリ部分は $O(P + Q \log P)$ ないし $O(P \log P + Q)$ で可能
- 全体では $O(M \log M + P + Q \log P)$ や $O(M \log M + P \log P + Q)$ で解けました
- $f(i) + i$ や $f(i) + A_i$ などを管理するテクニックはたまにみます

Subtask 6

$N, M \leq 8\,000$

6 後ろの小課題も見てみよう

- この小課題は小課題 4 で使った条件の言い換えなしで解けます
- 後ろの小課題が前の小課題と独立していて、そちらが解けたということがある
- 後ろの小課題や問題も見よう！

6

$f(l)$ を求める

- これまで同様に f を求めたいが l を P 通り試すと流石に大変
- そもそも P 通り試す必要はあるのか
- 例えば 2 本の辺について C が 4, 7 とあるとき 5 でも 6 でも 7 でも使える辺は変わらない
- つまり、 l はある辺の C として登場するものだけ考えれば OK
- これは $O(M)$ 通り

6

解法

- 小課題 3 で書いた尺取り法を使えば $O((N + M)M)$ で f が分かる
- 次はクエリに答えるところ
- 小課題 5 では $g(x) = f(x) - x$ として並べた
- 同様に g を求めて並べる
- 並べた中で何番目まで見ればいいかなどは同様にできる
- 座標圧縮をしている感じ

- 実装するときにはどの値が何番目かを管理しておく と 便利
- 全体としては $O((N + M)M + Q\log M)$ など で 解ける
- 各 l について r の最小値を二分探索して $O((N + M)M\log M)$ や $O((N + M)M\log P)$ など がつくと間に合わないと思われる
- クエリあたり全部見て $O(MQ)$ が ついても間に合わないと思われる

Subtask 7

追加制約なし

7 小課題 5 と小課題 6 の復習

- 小課題 5 で f を求めるための言い換えと f を求めてからクエリに答えるところをやった
- 小課題 6 で座圧できることがわかった
- これを合体すれば OK
- 全体で $O((M + Q)\log M)$ で解けました

7

別解

- 交換をすることである i について $[i, f(i)]$ を含むようにしたい
- $[L, R]$ から伸ばすことを考える
- ① r を増やすだけ = 右端を伸ばす
- $f(l)$ を求めた l のうち L 以上の中で最小のものに着目すれば OK
- ② l を減らすだけ = 左端を伸ばす
- $f(l) \leq r$ なる l のうち最大のが分かれば OK

7

別解

- ③ l を減らして r も増やす = 左端も右端も伸ばす
- このとき伸ばした後の区間は含みたい区間のどれかに一致
- そうでないなら余計に伸ばしたことになる
- ではどの区間に一致させるか
- $[i, f(i)]$ に一致させるとき $i \leq L \leq R \leq f(i)$ なので伸ばしたのは
(伸ばしたあとの長さ) - (最初の長さ) = $(f(i) - i) - (R - L)$

7 別解

- 結局ある範囲における $f(i) - i$ を求める形になる
- 計算量は変わらず $O((M + Q)\log M)$ になりました
- 機械的に式変形しても別解のように区間を伸ばす形にしても結論は同じようになる

7

間違いについて

- 考察をすると区間を伸ばして与えられた区間のうちどれかを含むようにしたいという問題になった
- 与えられた区間を左端で sort して $T_1, T_2, \dots, T_n$ とでも呼ぶ
- $T_i = [l_i, r_i]$ とでもして伸ばす区間を $S = [L, R]$ としよう
- $f(i) = S$ から T_i に持っていくためのコストとして求めたいのは $\min_i(f(i))$
- ここで予想「 $f(i)$ は三分探索によって極値が計算できる」を立てる 実際書いてみると通らない

7

間違いについて

- 実はこの予想は間違い 証明をする
- $M = 2N - 3$ として辺を C が小さい順に sort したとき B が $(2, 3, \dots, N, 2, 3, \dots, N - 1)$ になるとする
- このとき $T_i = [C_i, C_{i+N-2}]$ になる
- $N = 6$ として具体例を構築する

7

間違いについて

i	1	2	3	4	5	6	7	8	9
B	2	3	4	5	6	2	3	4	5
C	3	5	$X - 7$	$X - 5$	X	$X + 3$	$X + 4$	$2X - 1$	$2X$

- 大きな数 X を用いて以下のようにしたとする
- $S = [X, X]$ とする
- T は $[3, X], [5, X + 3], [X - 7, X + 4], [X - 5, 2X - 1], [X, 2X]$
- 考察より T の長さが大事 これは $(X - 3, X - 2, 11, X + 4, X)$

7

間違いについて

- $(X - 3, X - 2, 11, X + 4, X)$ という列から最小値を見つけるのは今回の三分探索では正当かもしれないが少なくともこの列は凸ではない
- なので三分探索は正当ではない

Step 8

得点分布

8

得点分布(高 3 含む)

点数	100	80	71	58	51	49	48	42	38	30	26	23	19	15	8	7
1	0	0	0	0	-	0	0	-	0	0	0	-	-	0	-	0
2	0	0	0	0	0	0	0	0	0	-	0	-	0	0	0	-
3	0	0	0	0	0	0	0	0	-	-	0	-	0	-	-	-
4	0	0	0	0	0	0	-	0	0	0	-	0	-	-	-	-
5	0	0	-	0	0	-	-	-	-	-	-	-	-	-	-	-
6	0	0	0	-	-	-	0	-	-	-	-	-	-	-	-	-
7	0	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
人数	33	2	1	1	1	10	4	1	2	4	14	1	1	18	1	26
累計	33	35	36	37	38	48	52	53	55	59	73	74	75	93	94	120

JOI 2024/25 本選 4

長いだけのネクタイ 2
(Just Long Neckties 2)

解説：渡邊雄斗 (yuto1115)

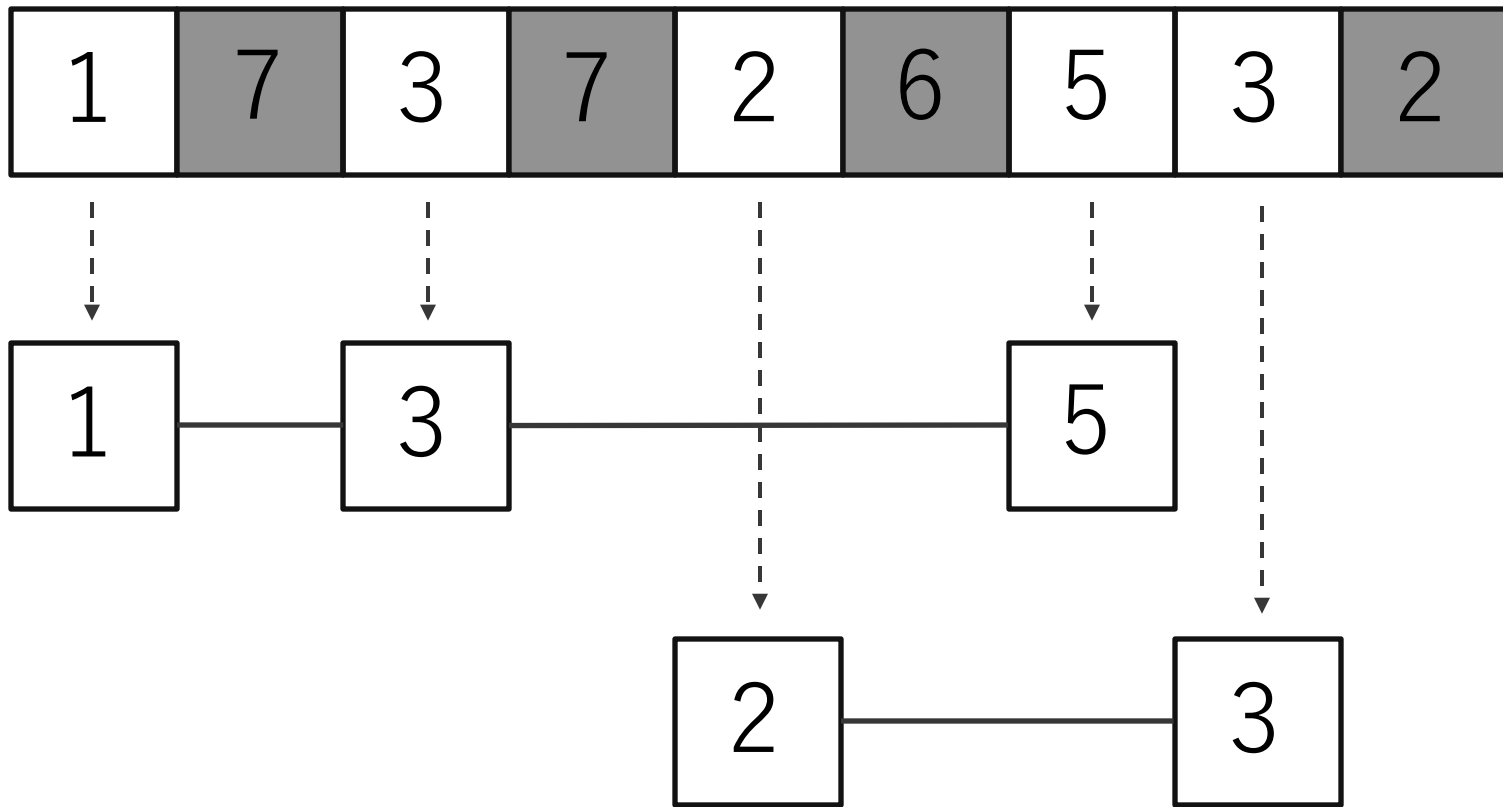
問題概要

問題概要

- 長さ N の数列 $A = (A_1, A_2, \dots, A_N)$ が与えられる
- まず、数列のいくつかの要素を選んで消す
 - ただし、連続する要素をどちらも消してはいけない
- 残る要素をいくつかの広義単調増加部分列に分割する
- 最小でいくつの部分列に分割できるか？

1	7	3	7	2	6	5	3	2
---	---	---	---	---	---	---	---	---

1	7	3	7	2	6	5	3	2
---	---	---	---	---	---	---	---	---



考察 1: 初歩的な DP

小課題 2 $N \leq 500, A_i \leq 2$ (6 pts.)

小課題 3 $N \leq 500, A_i \leq 5$ (12 pts.)

考察 1: 初歩的な DP

- まずは答えの範囲を考えたい
 - 最小でも 1 は必要 ($N \geq 2$)
 - $L = \max A_i$ として、 L あれば必ず可能
(値が i である要素をまとめて i 番目の列とする)

考察 1: 初歩的な DP

小課題 2 $N \leq 500, A_i \leq 2$

- 答えは 1 or 2 なので、1 が可能かどうか調べたい
 - 残る要素が広義単調増加となるように消す要素を選べるか？
- $dp_i =$ 「 i 番目まで決めた (i 番目は残した) とき、
残した要素が単調増加」が可能かどうか (true / false)

考察 1: 初歩的な DP

小課題 2 $N \leq 500, A_i \leq 2$

- dp_i = 「 i 番目まで決めた (i 番目は残した) とき、
残した要素が単調増加」が可能かどうか (true / false)
- 初期条件: $dp_0 = \text{true}$
- 遷移: $dp_i = \text{true}$ かつ $A_i \leq A_{i+1}$ なら $dp_{i+1} = \text{true}$
 $dp_i = \text{true}$ かつ $A_i \leq A_{i+2}$ なら $dp_{i+2} = \text{true}$

考察 1: 初歩的な DP

小課題 2 $N \leq 500, A_i \leq 2$

- $O(N)$ の DP で解くことができる
- (別解)

答えが 2

$\Leftrightarrow A_i = A_{i+1} = 2, A_j = A_{j+1} = 1$ なる $i < j$ が存在

考察 1: 初歩的な DP

小課題 3 $N \leq 500, A_i \leq 5$

- 答えは 1 以上 5 以下
- 先頭要素から見ていって、残すかどうかを決めつつ、
同時に部分列を構築していく

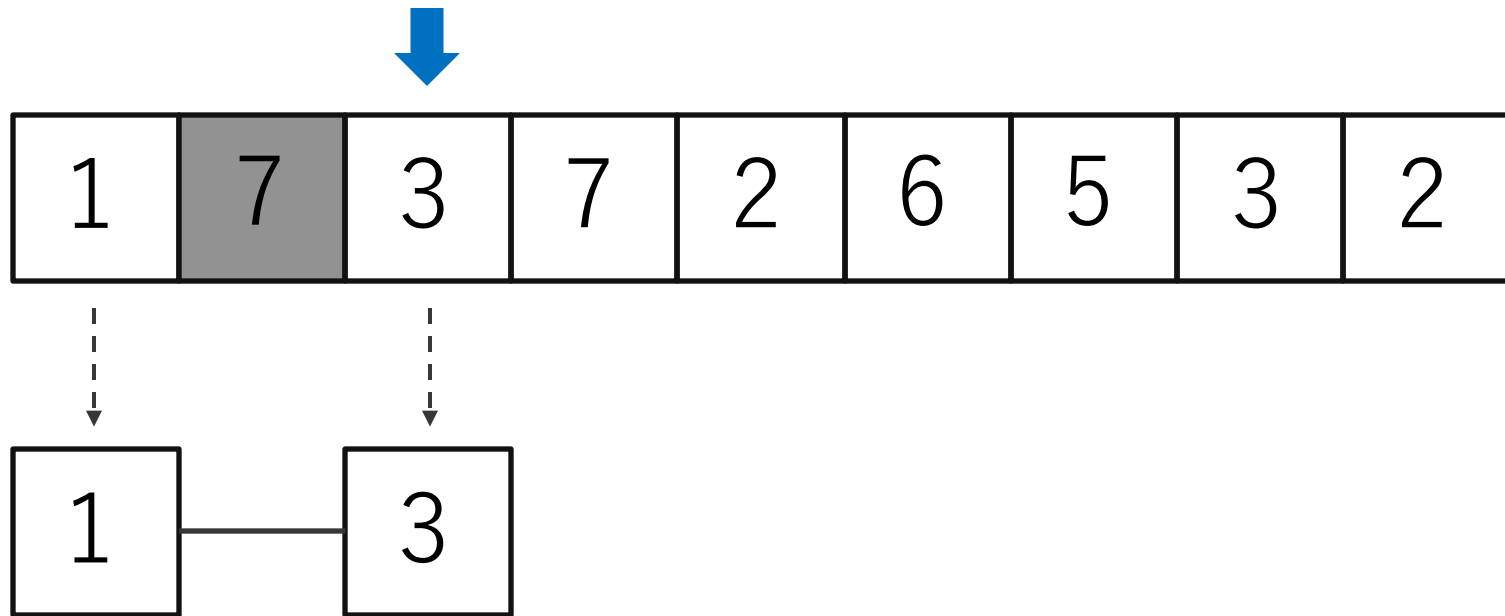
1	7	3	7	2	6	5	3	2
---	---	---	---	---	---	---	---	---

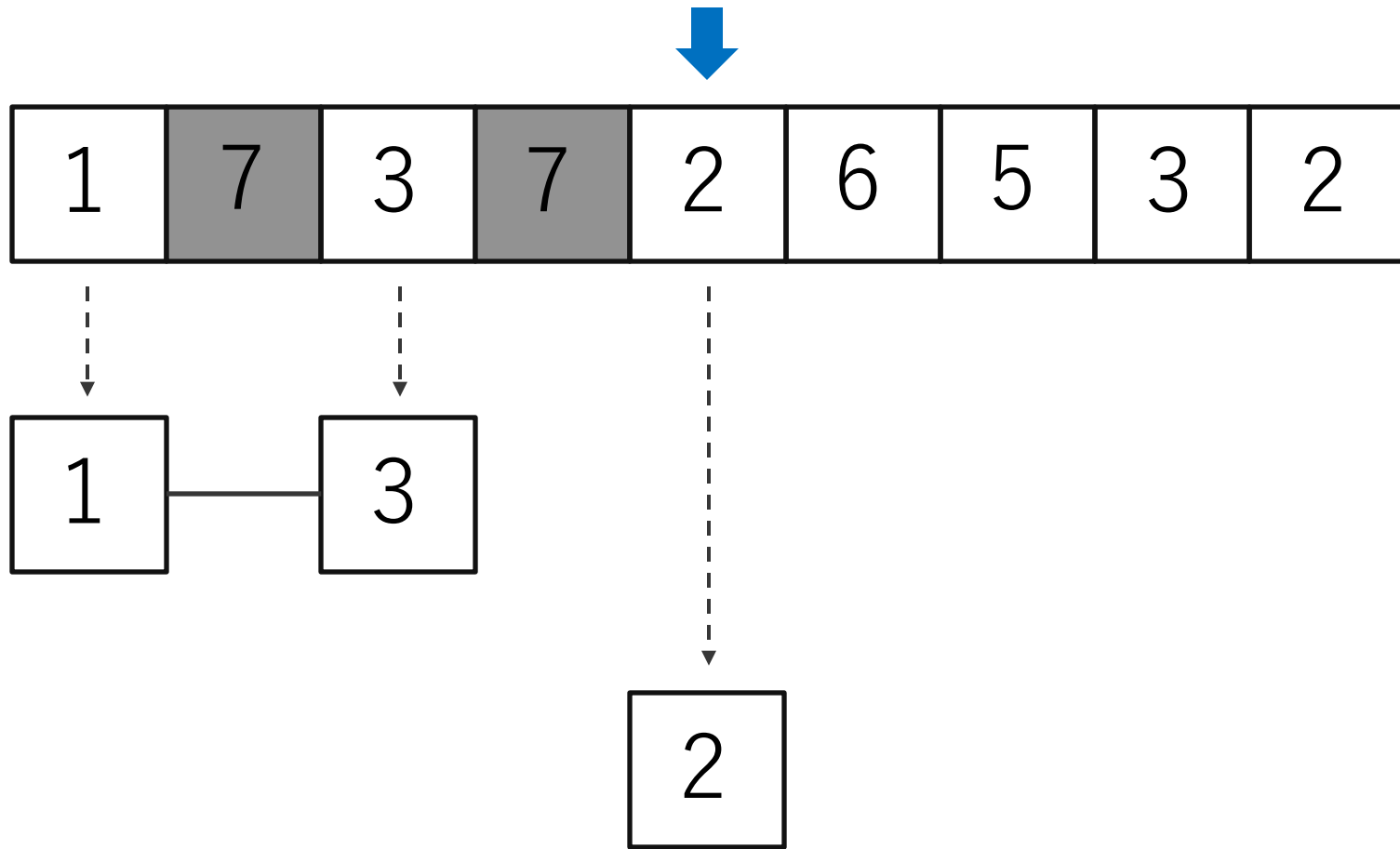


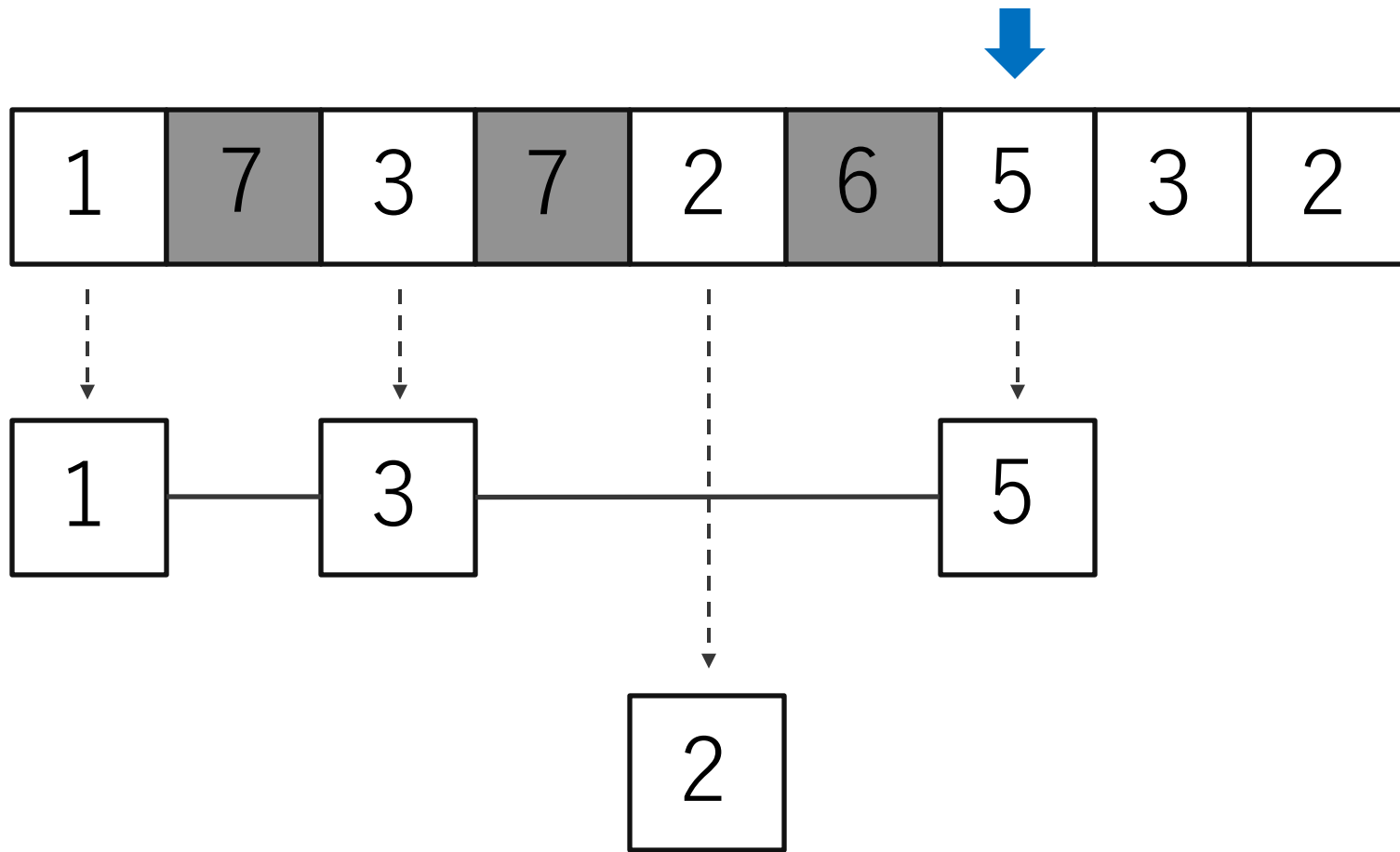
1	7	3	7	2	6	5	3	2
---	---	---	---	---	---	---	---	---

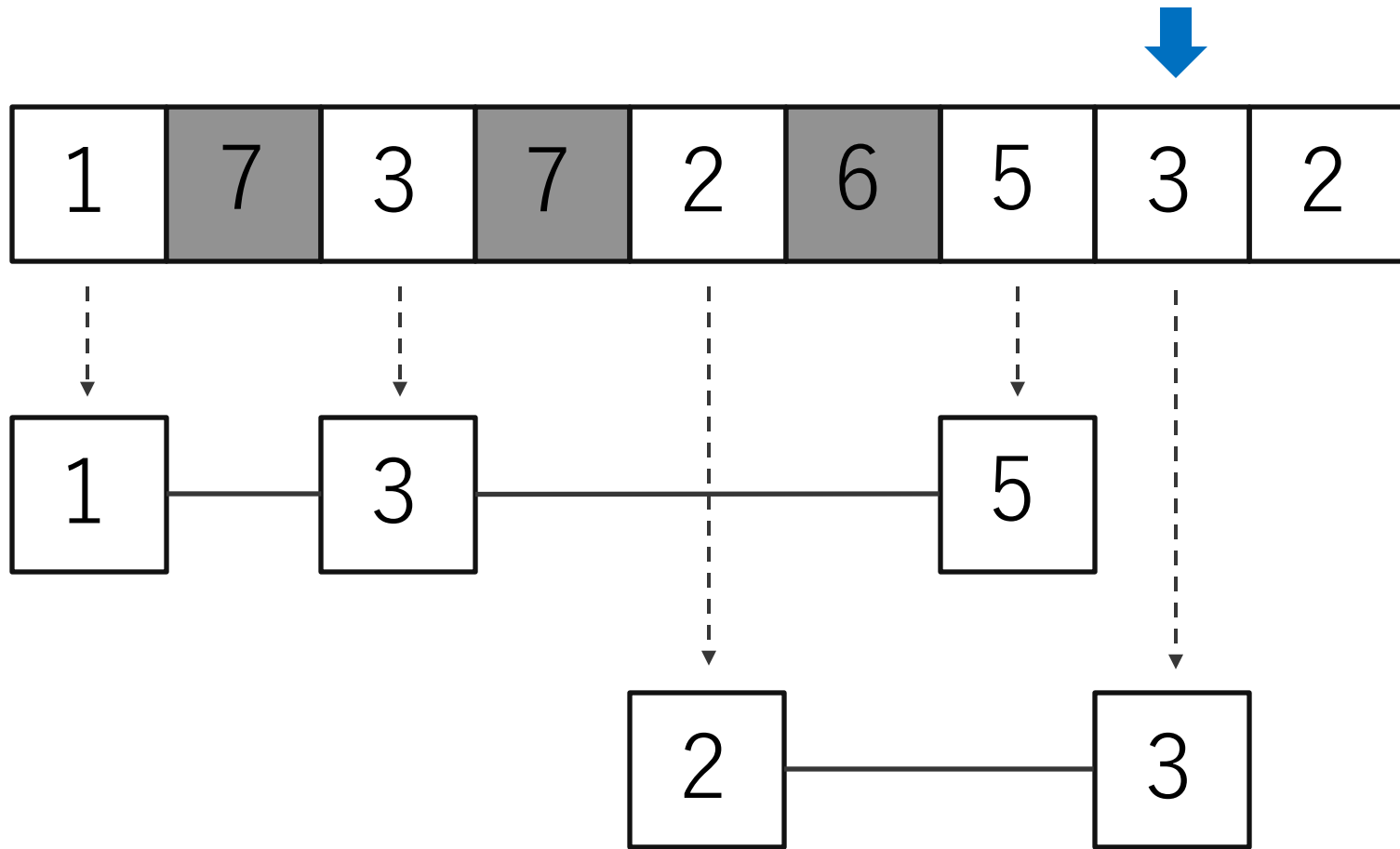


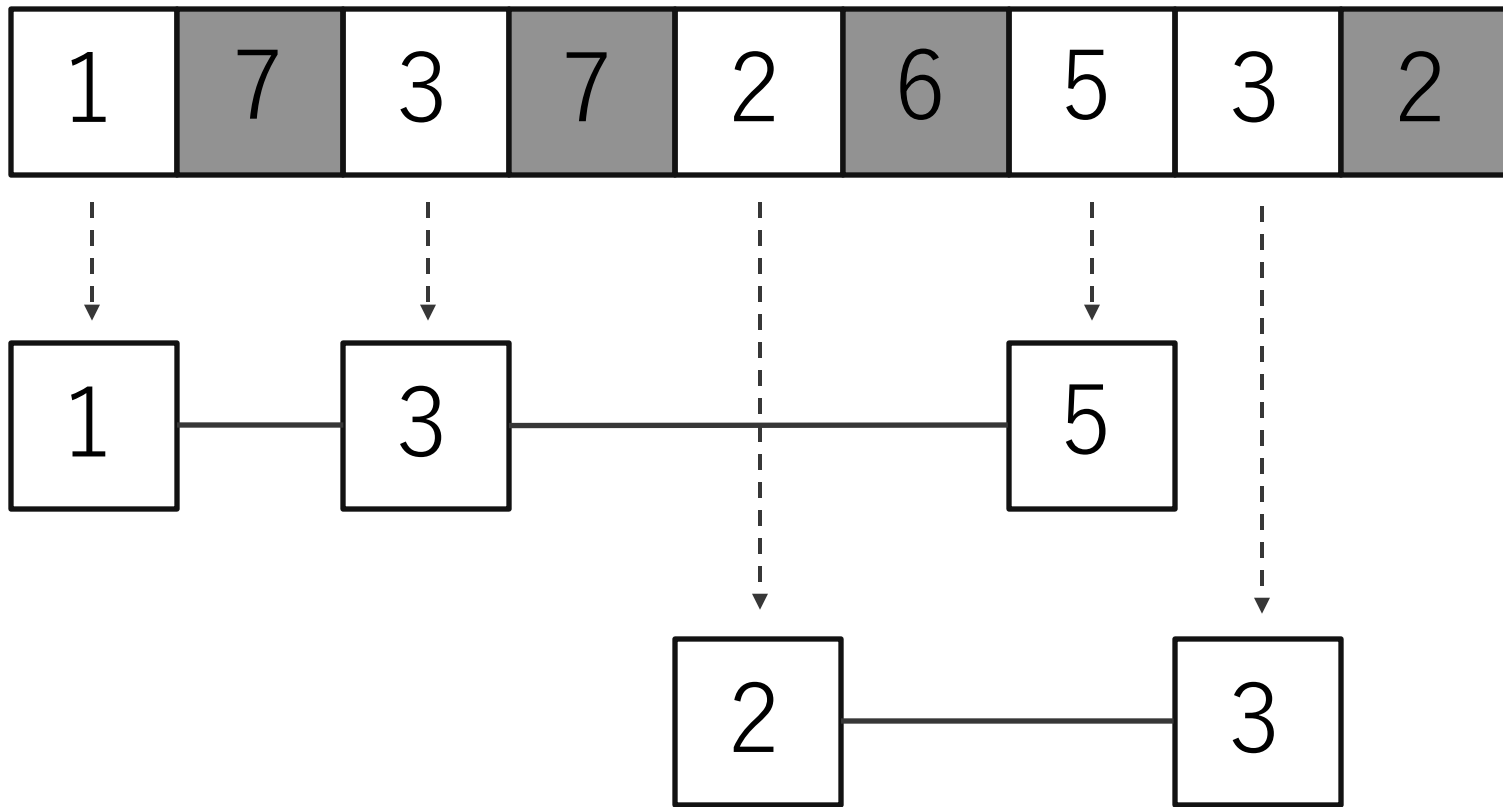
1

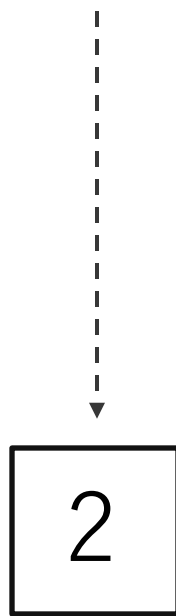
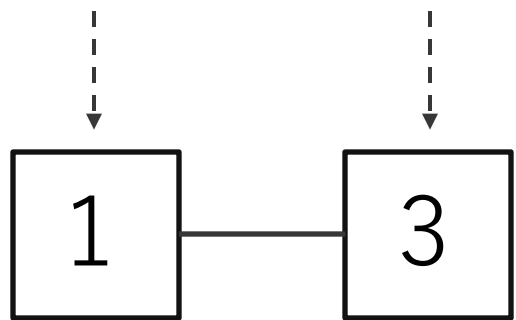
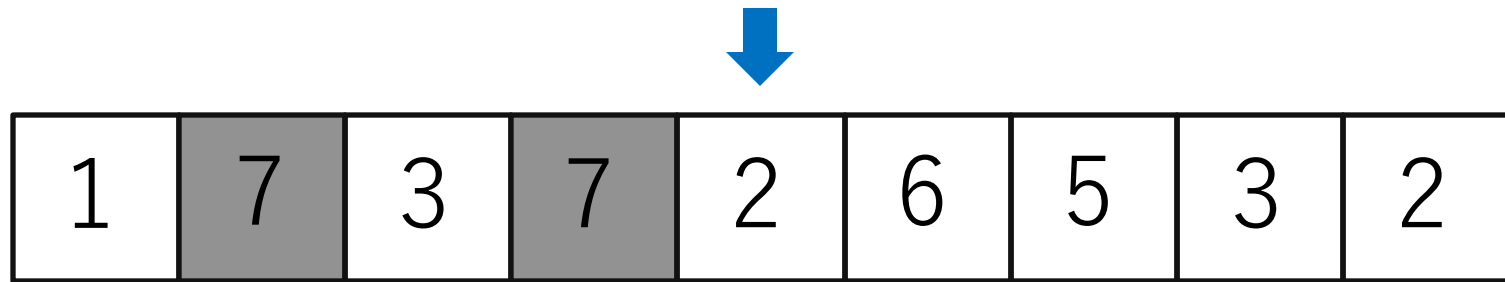




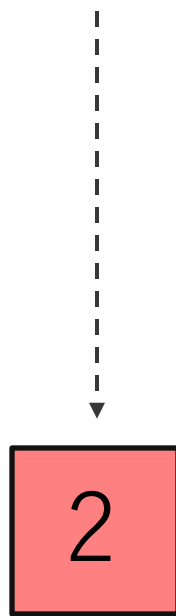
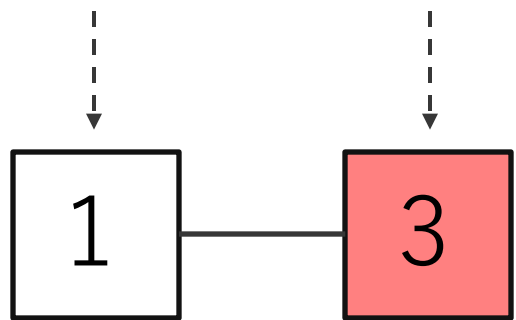
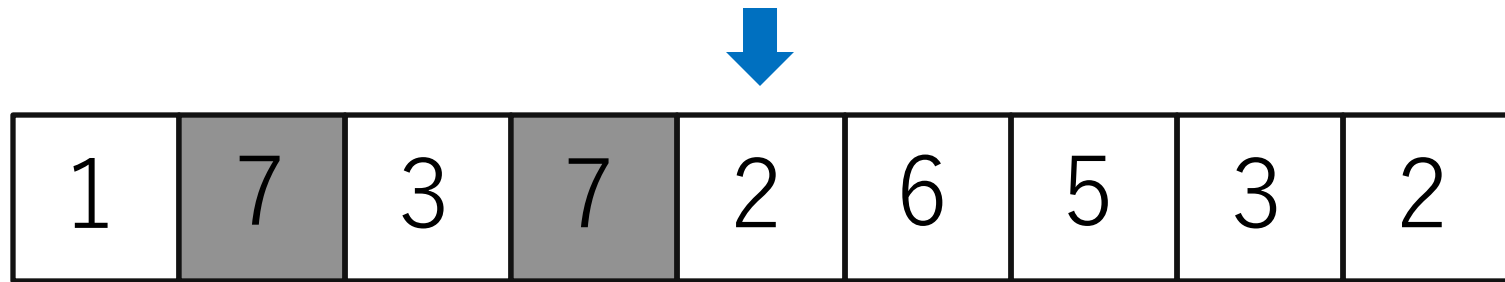








必要な情報は？



今見ているところ
各列の先頭

考察 1: 初歩的な DP

小課題 3 $N \leq 500, A_i \leq 5$

- 列は最大 5 個あれば十分
- $dp_{i,p,q,r,s,t}$ = 「 i 番目まで決めた (i 番目は残した) とき、列 1,2,3,4,5 の先頭がそれぞれ p, q, r, s, t 」は可能かどうか (true / false)
- 空の列は先頭が 0 ということにしておいて、
最終的に先頭が 0 の列の個数が最大のものが最適

考察 1: 初歩的な DP

小課題 3 $N \leq 500, A_i \leq 5$

- $dp_{i,p,q,r,s,t} =$ 「 i 番目まで決めた (i 番目は残した) とき、列 $1,2,3,4,5$ の先頭がそれぞれ p,q,r,s,t 」 は可能かどうか (true / false)
- 遷移: 次に残す要素 ($i+1$ or $i+2$) とその要素を足す列を全探索
- $L = \max A_i$ として、状態 $O(NL^L)$ 、遷移 $O(L)$

考察 2: 貪欲法

小課題 1 $N \leq 15, A_i \leq 21$ (10 pts.)

考察 2: 貪欲法

小課題 1 $N \leq 15, A_i \leq 21$

- N が小さいので、どの要素を残すかを $O(2^N)$ 通り全探索できる
- 残す要素が決まったとき、最小で何個の部分列に分割可能か求めたい
 - $L = \max A_i$ の値が大きいため DP はできない → 貪欲法
 - 追加できる列があるなら、そのうち先頭が最大のものに追加
 - 追加できる列がないなら、新しい列を作る

1	7	3	7	2	6	5	3	2
---	---	---	---	---	---	---	---	---

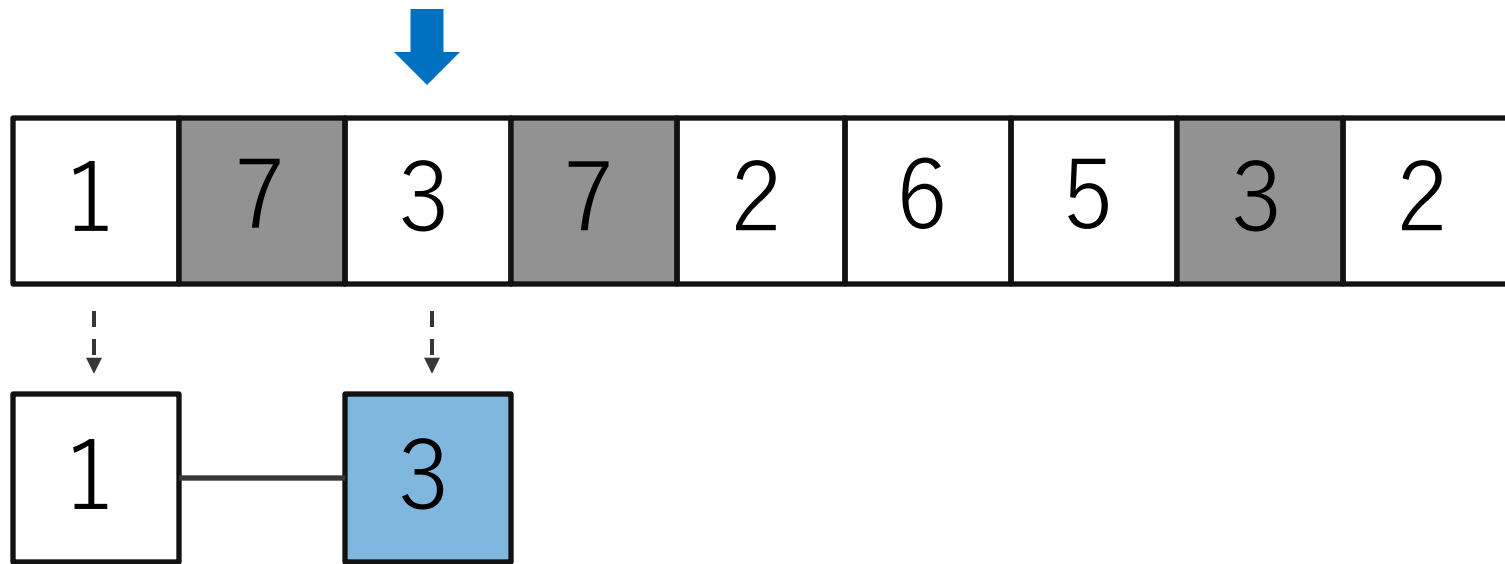
1	7	3	7	2	6	5	3	2
---	---	---	---	---	---	---	---	---

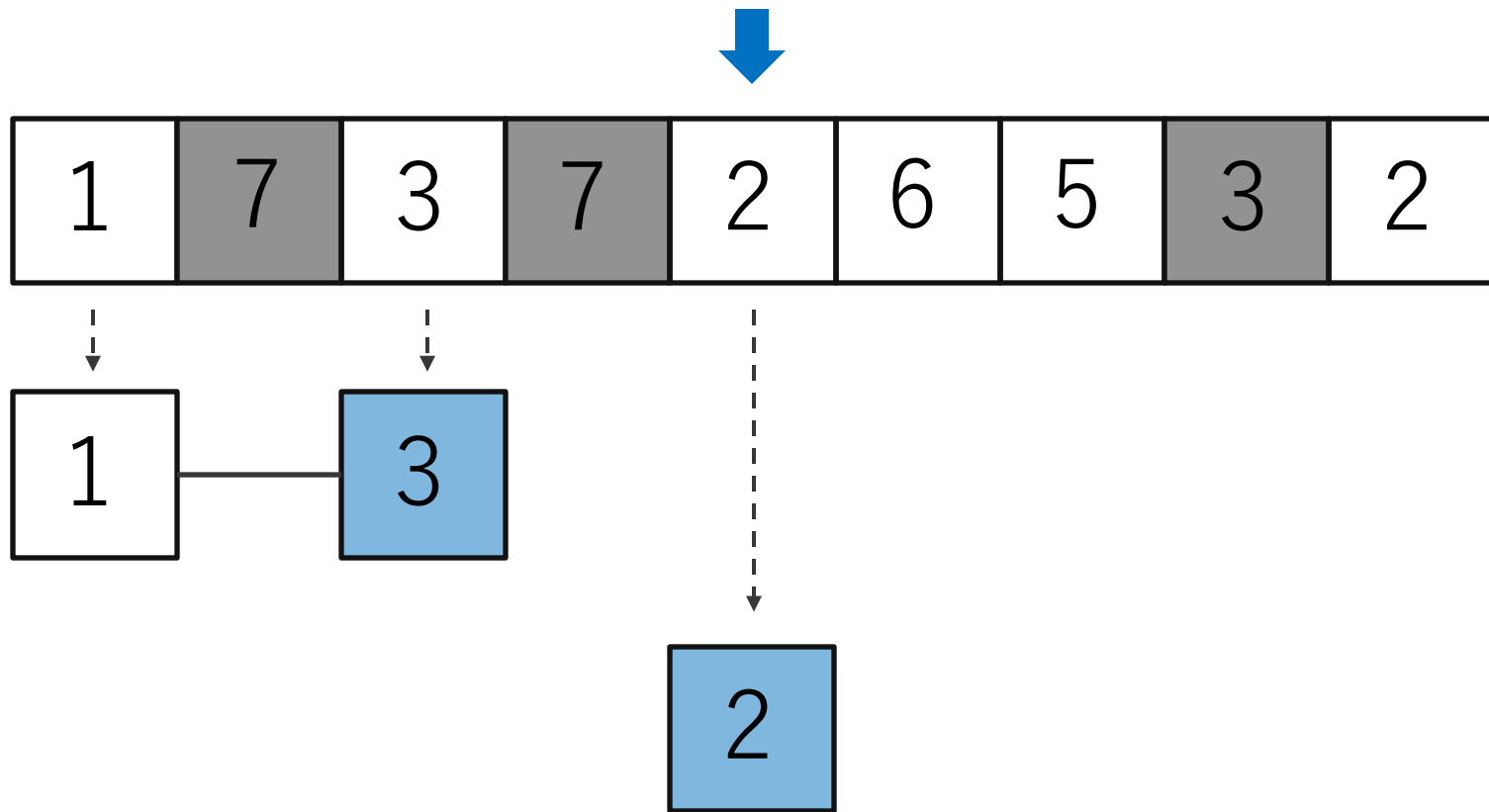


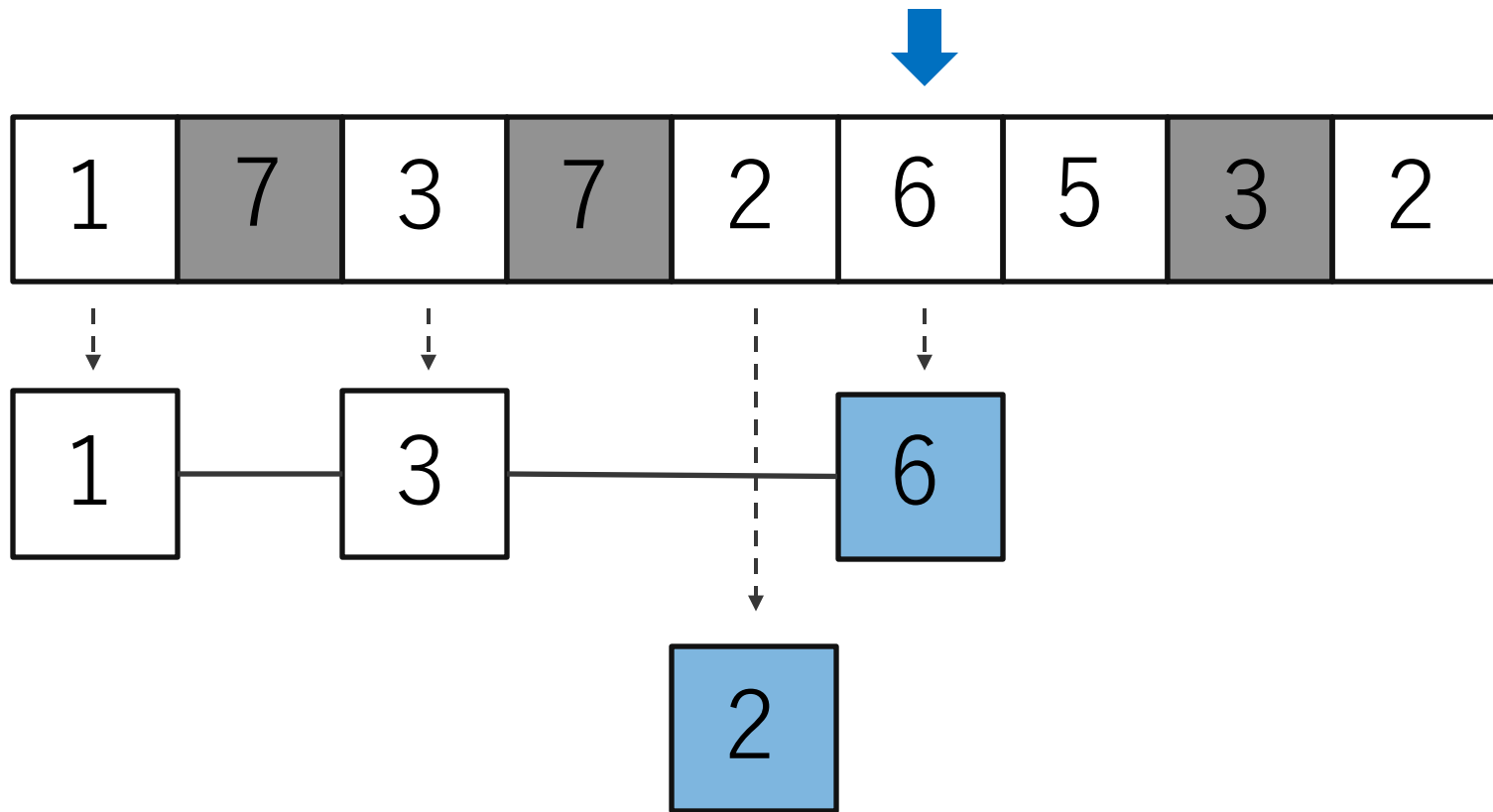
1	7	3	7	2	6	5	3	2
---	---	---	---	---	---	---	---	---

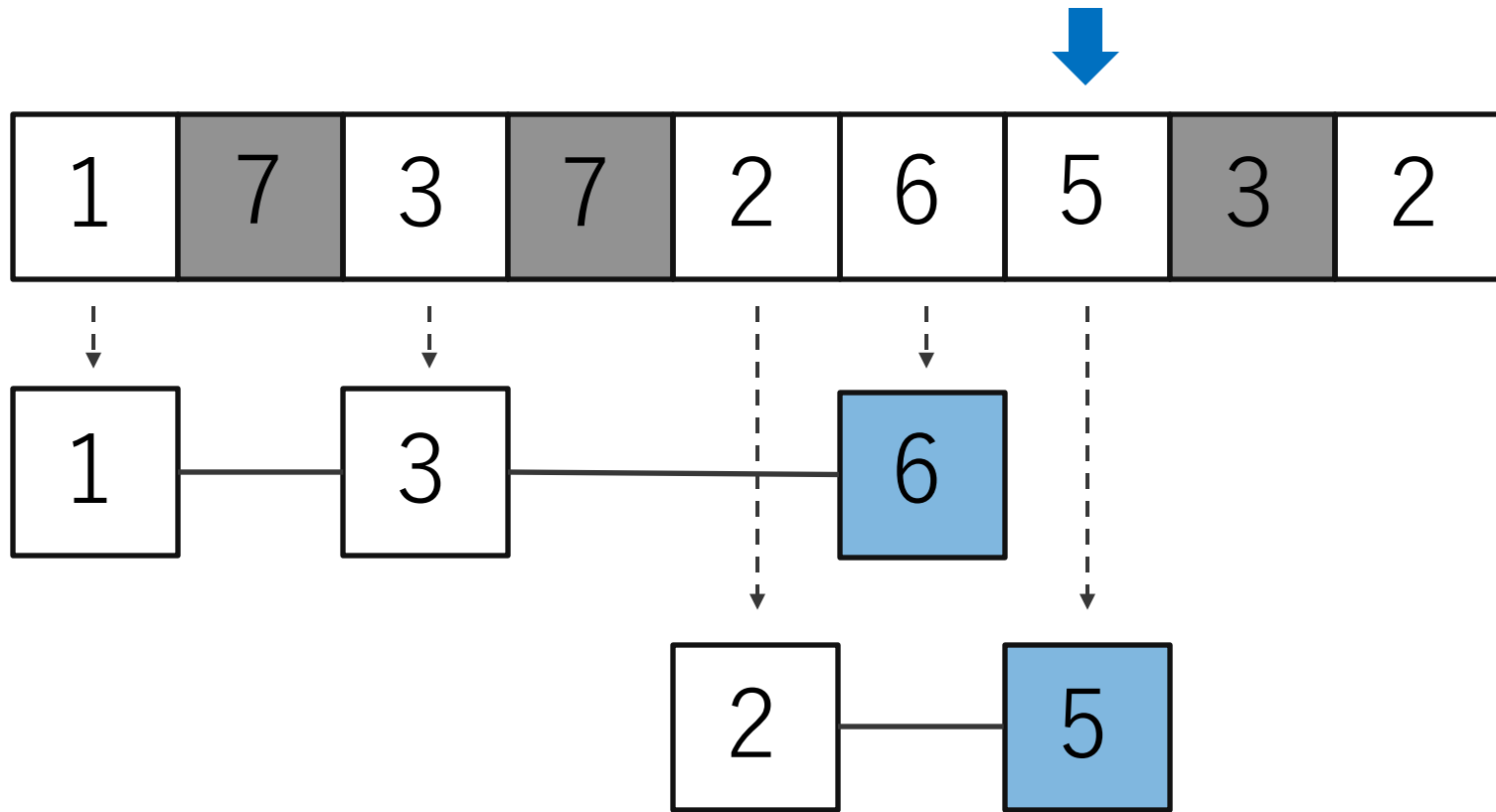


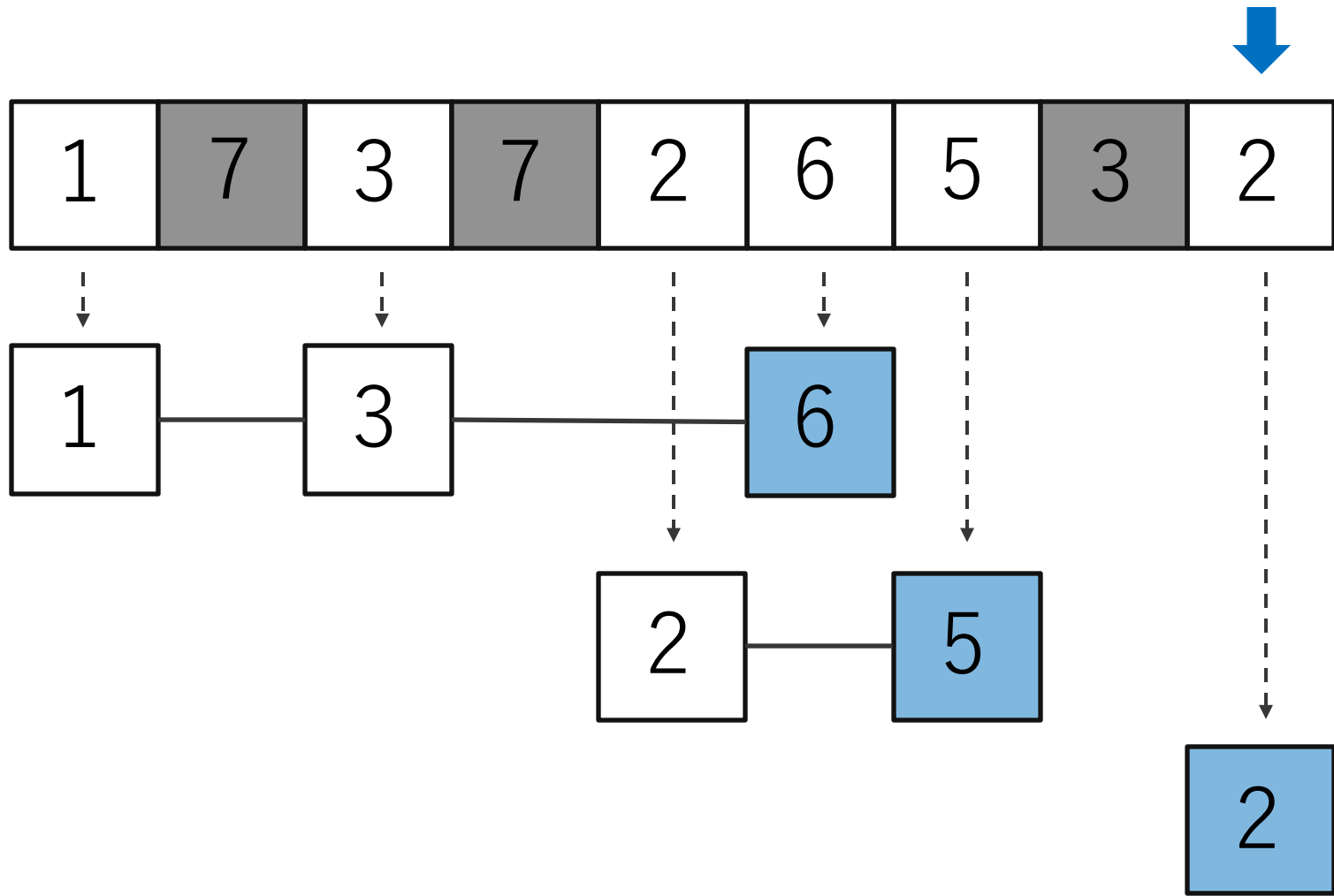
1











考察 2: 貪欲法

小課題 1 $N \leq 15, A_i \leq 21$

- $O(2^N)$ 通り全探索 $\rightarrow$ 貪欲法で各 $O(N^2)$ など

考察 3: bit DP

小課題 4 $N \leq 500, A_i \leq 15$ (18 pts.)

考察 3: bit DP

小課題 4 $N \leq 500, A_i \leq 15$

- $O(NL^L)$ 状態の DP に、先ほどの貪欲法を統合する
 - 元々の遷移：次に残す要素を決めた上で、追加する列を全探索
 - 改善後：次に残す要素を決めたとき、追加する列を**貪欲法で決定**
- 状態数は結局変わらない…？

考察 3: bit DP

小課題 4 $N \leq 500, A_i \leq 15$

- $O(NL^L)$ 状態の DP に、先ほどの貪欲法を統合する
- 観察：貪欲法を用いた場合、どの途中状態においても、
各列の先頭要素はすべて相異なる
- 列の先頭要素は $\{1, 2, \dots, L\}$ の部分集合とみなせ、 2^L 通り

考察 3: bit DP

小課題 4 $N \leq 500, A_i \leq 15$

- $dp_{i,S}$ = 「 i 番目まで決めた (i 番目は残した) とき、各列の先頭要素として現れる値の集合が S 」は可能かどうか (true / false)
- 状態 $O(N2^L)$ 、遷移 $O(L)$ の bit DP
- 自然な実装をすれば全体でも計算量 $O(N2^L)$

考察 4: DP 高速化

小課題 5 $N \leq 5 \times 10^5, A_i \leq 15$ (26 pts.)

小課題 6 $N \leq 5 \times 10^5, A_i \leq 21$ (10 pts.)

小課題 7 $N \leq 5 \times 10^6, A_i \leq 21$ (18 pts.)

(満点)

考察 4: DP 高速化

- 小課題 4 で用いた bit DP の復習

$$A = \begin{array}{|c|c|c|c|c|c|c|} \hline 2 & 1 & 1 & 2 & 2 & 1 & 2 \\ \hline \end{array}$$

$S \setminus i$	0	1	2	3	4	5	6	7
$\{\}$								
$\{1\}$								
$\{2\}$								
$\{1, 2\}$								

$$A = \begin{array}{|c|c|c|c|c|c|c|} \hline 2 & 1 & 1 & 2 & 2 & 1 & 2 \\ \hline \end{array}$$

$S \setminus i$	0	1	2	3	4	5	6	7
$\{\}$								
$\{1\}$								
$\{2\}$								
$\{1, 2\}$								

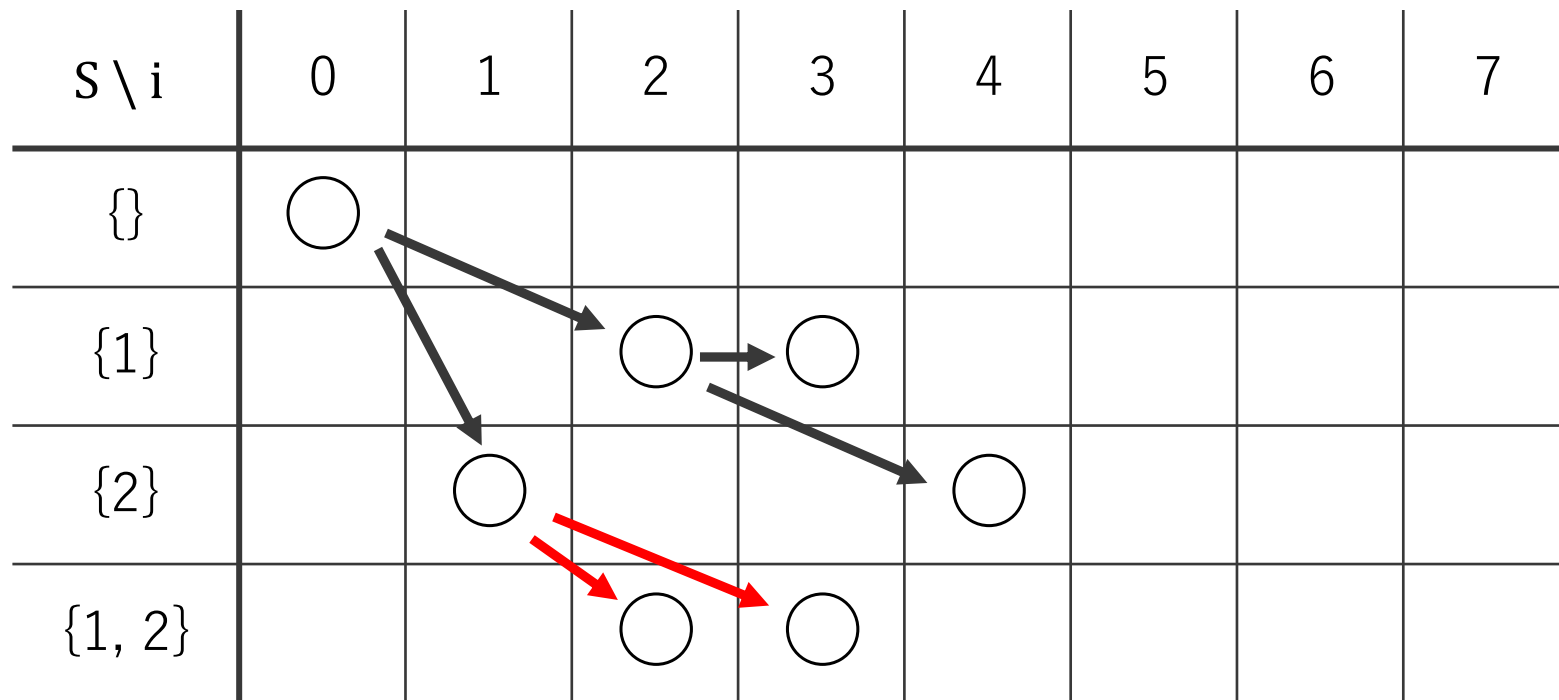
$$A = \begin{bmatrix} 2 & 1 & 1 & 2 & 2 & 1 & 2 \end{bmatrix}$$

$S \setminus i$	0	1	2	3	4	5	6	7
$\{\}$	○							
$\{1\}$			○					
$\{2\}$		○						
$\{1, 2\}$								

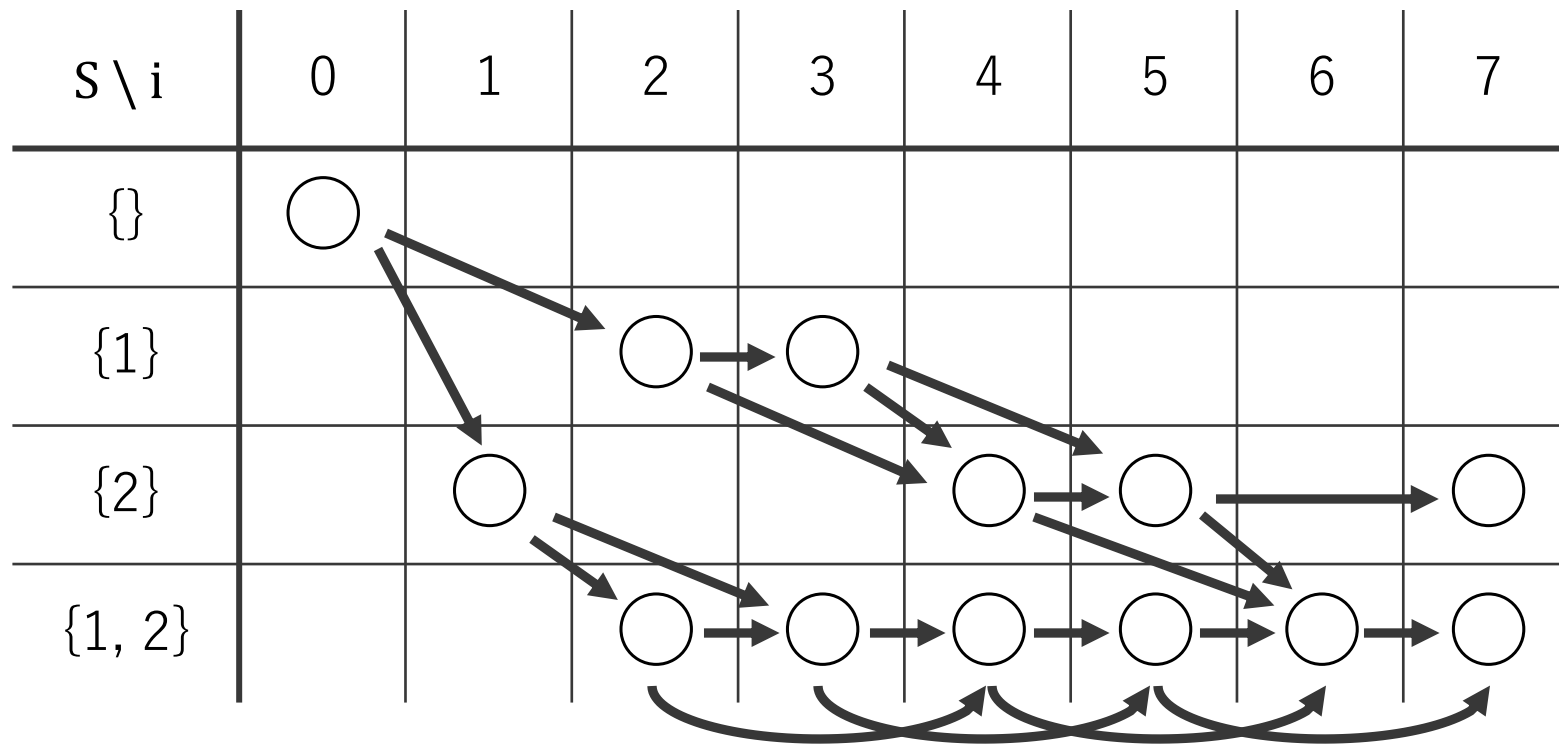
$$A = \begin{bmatrix} 2 & 1 & 1 & 2 & 2 & 1 & 2 \end{bmatrix}$$

$S \setminus i$	0	1	2	3	4	5	6	7
$\{\}$	○							
$\{1\}$			○	○				
$\{2\}$		○			○			
$\{1, 2\}$								

$$A = \begin{bmatrix} 2 & 1 & 1 & 2 & 2 & 1 & 2 \end{bmatrix}$$



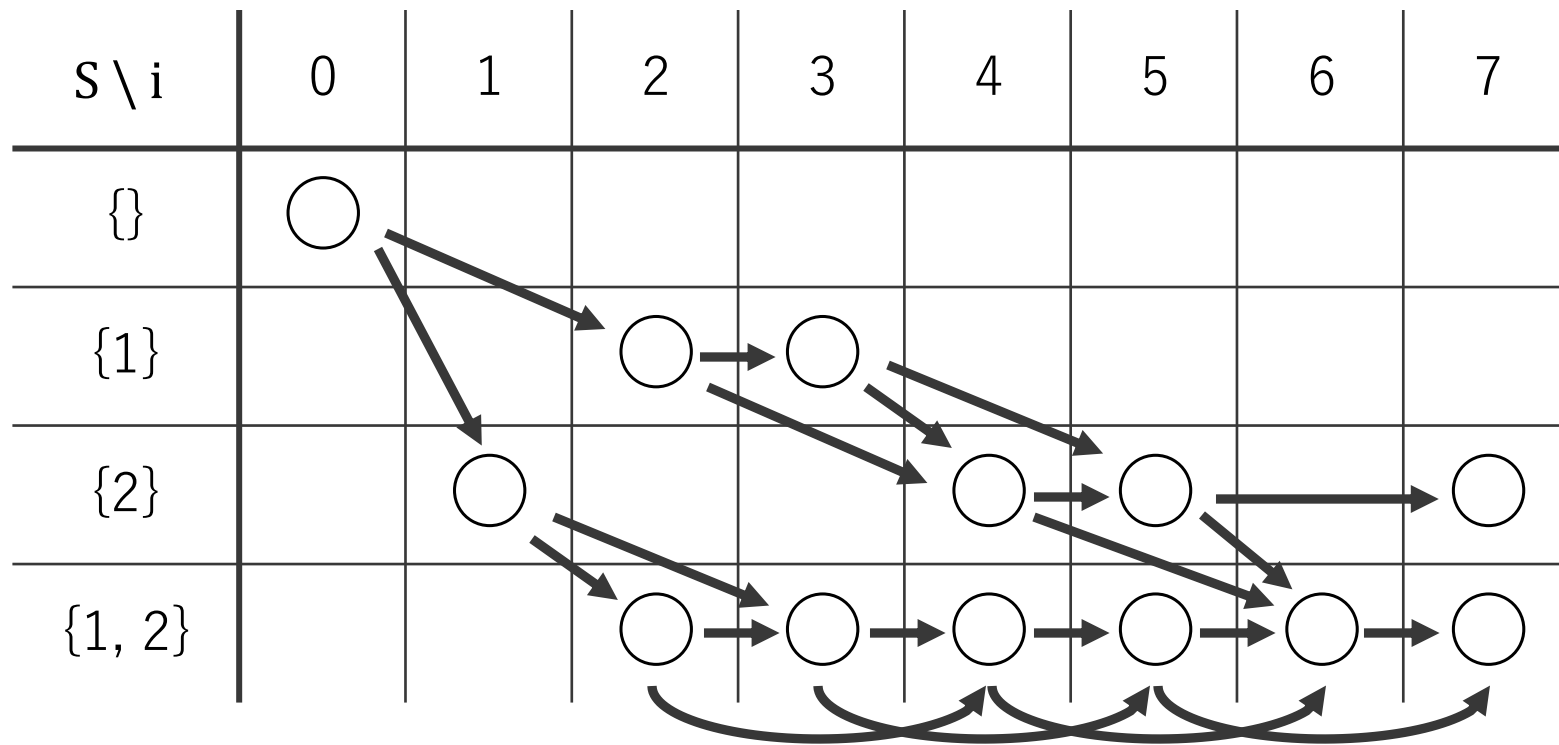
$$A = \begin{bmatrix} 2 & 1 & 1 & 2 & 2 & 1 & 2 \end{bmatrix}$$



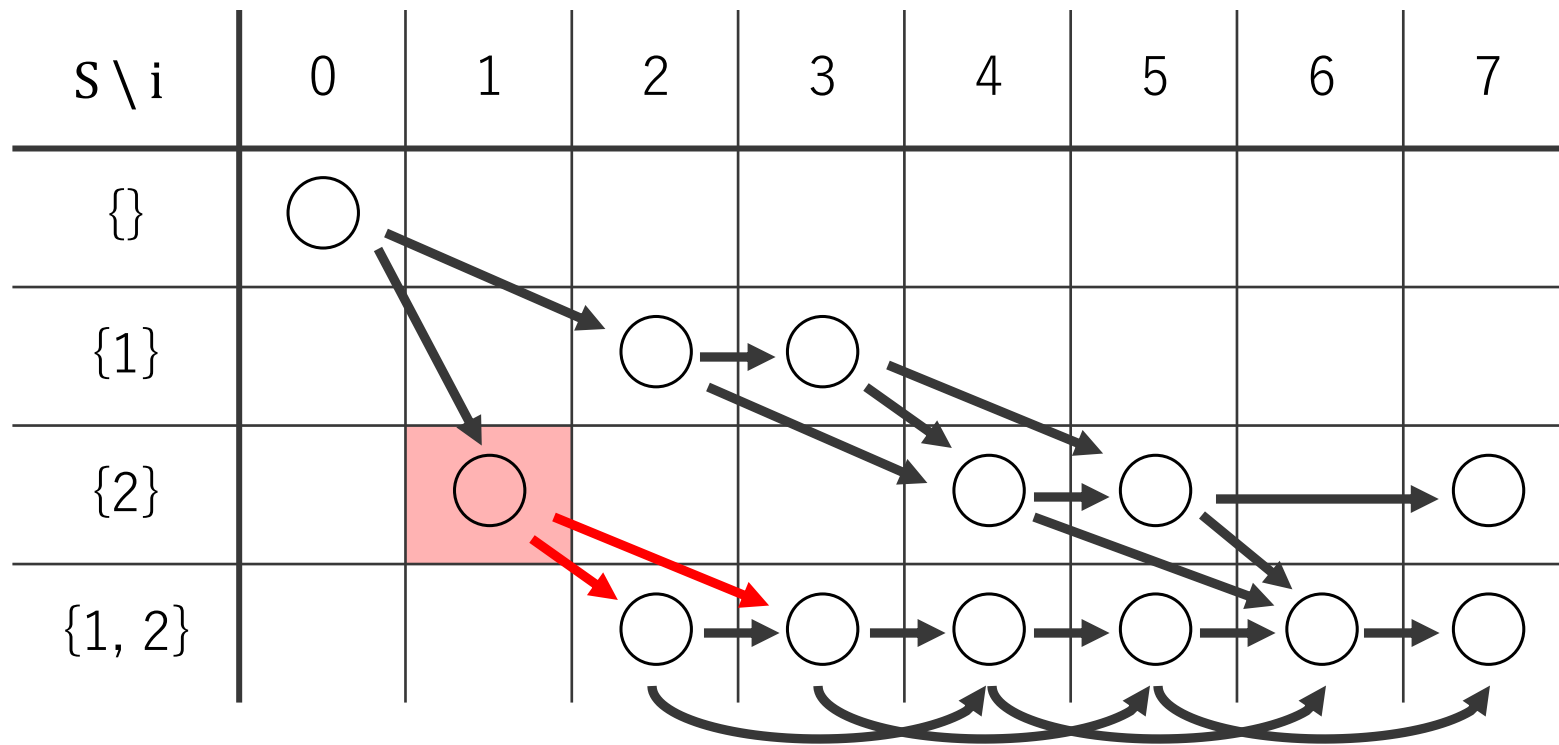
考察 4: DP 高速化

- 小課題 4 で用いた bit DP の復習
 - $N2^L$ 個の bool 値を保持している
 - 保持する情報の量を減らしたい
 - 不要な情報 はないだろうか？

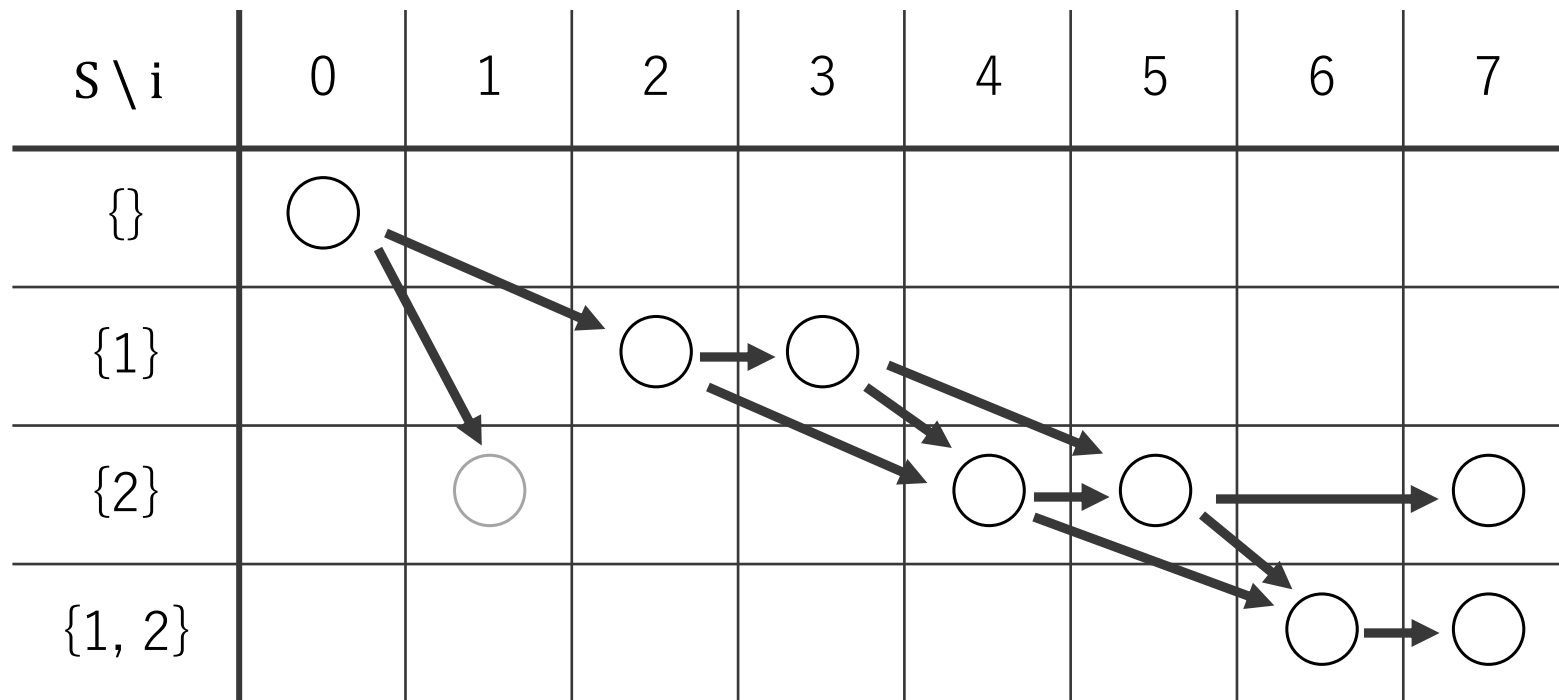
$$A = \begin{bmatrix} 2 & 1 & 1 & 2 & 2 & 1 & 2 \end{bmatrix}$$



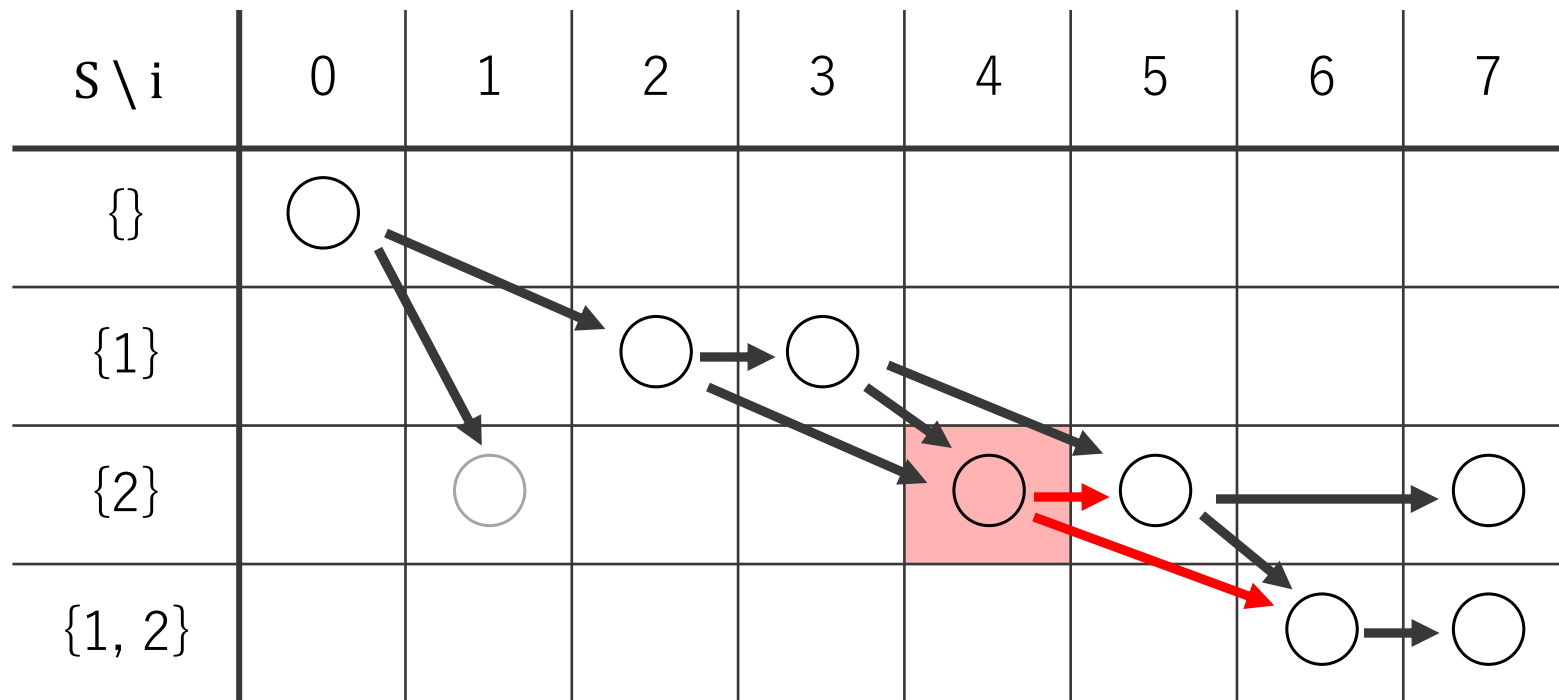
$$A = \begin{bmatrix} 2 & 1 & 1 & 2 & 2 & 1 & 2 \end{bmatrix}$$



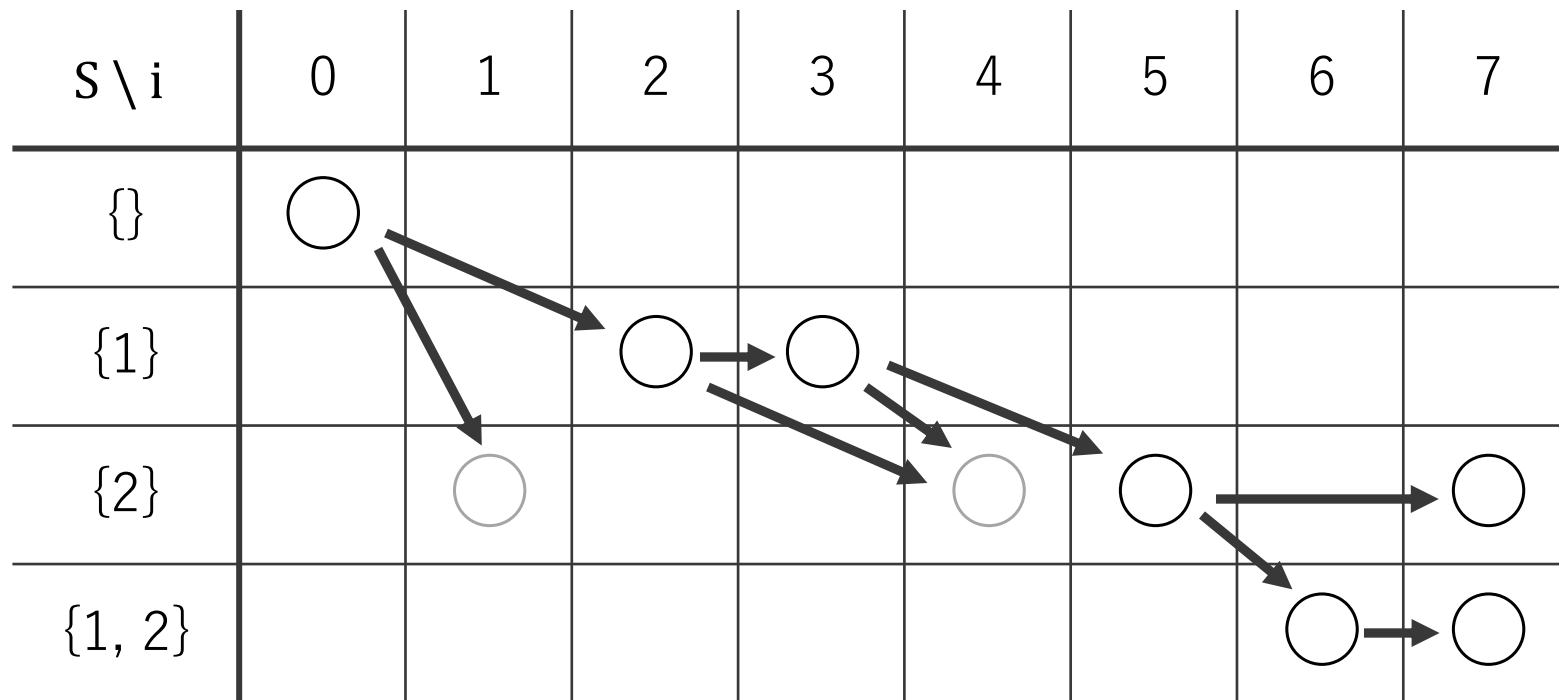
$$A = \begin{bmatrix} 2 & 1 & 1 & 2 & 2 & 1 & 2 \end{bmatrix}$$



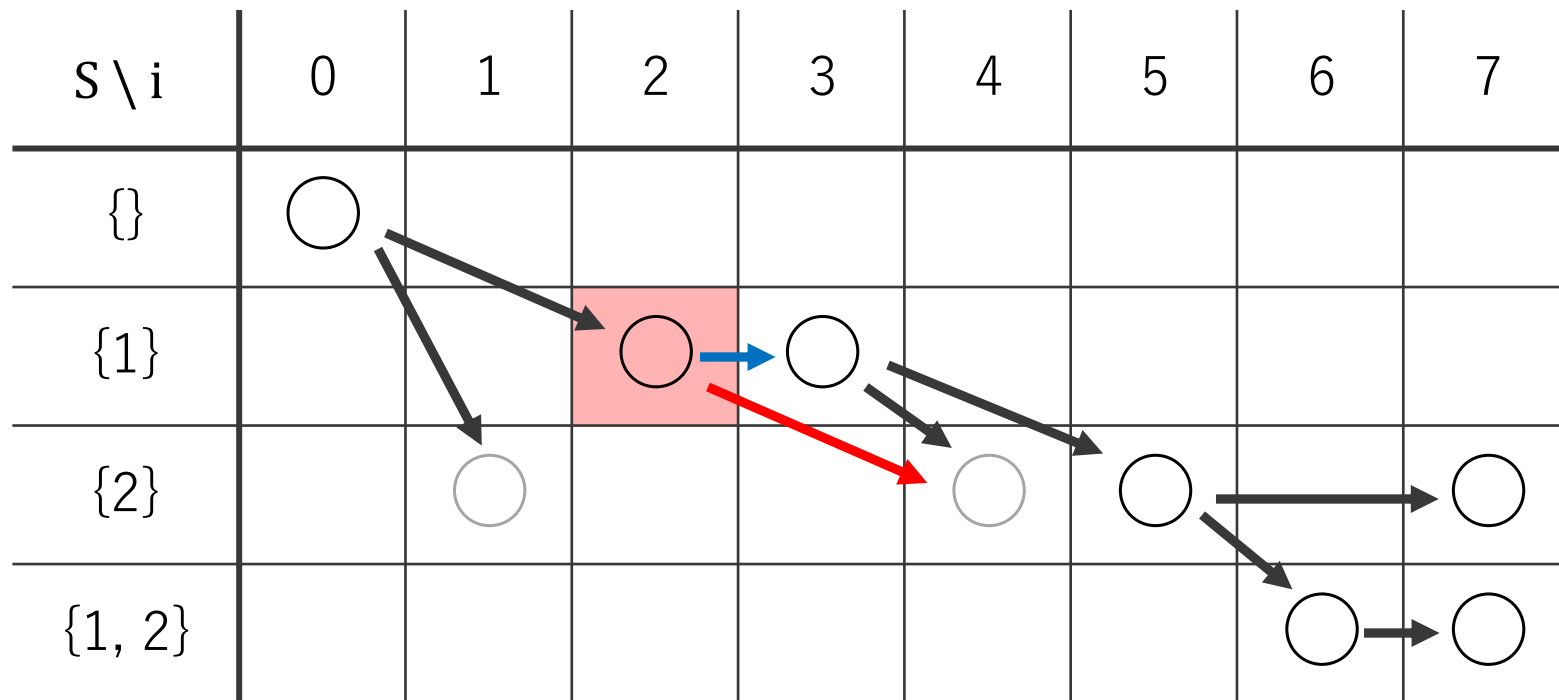
$$A = \begin{bmatrix} 2 & 1 & 1 & 2 & 2 & 1 & 2 \end{bmatrix}$$



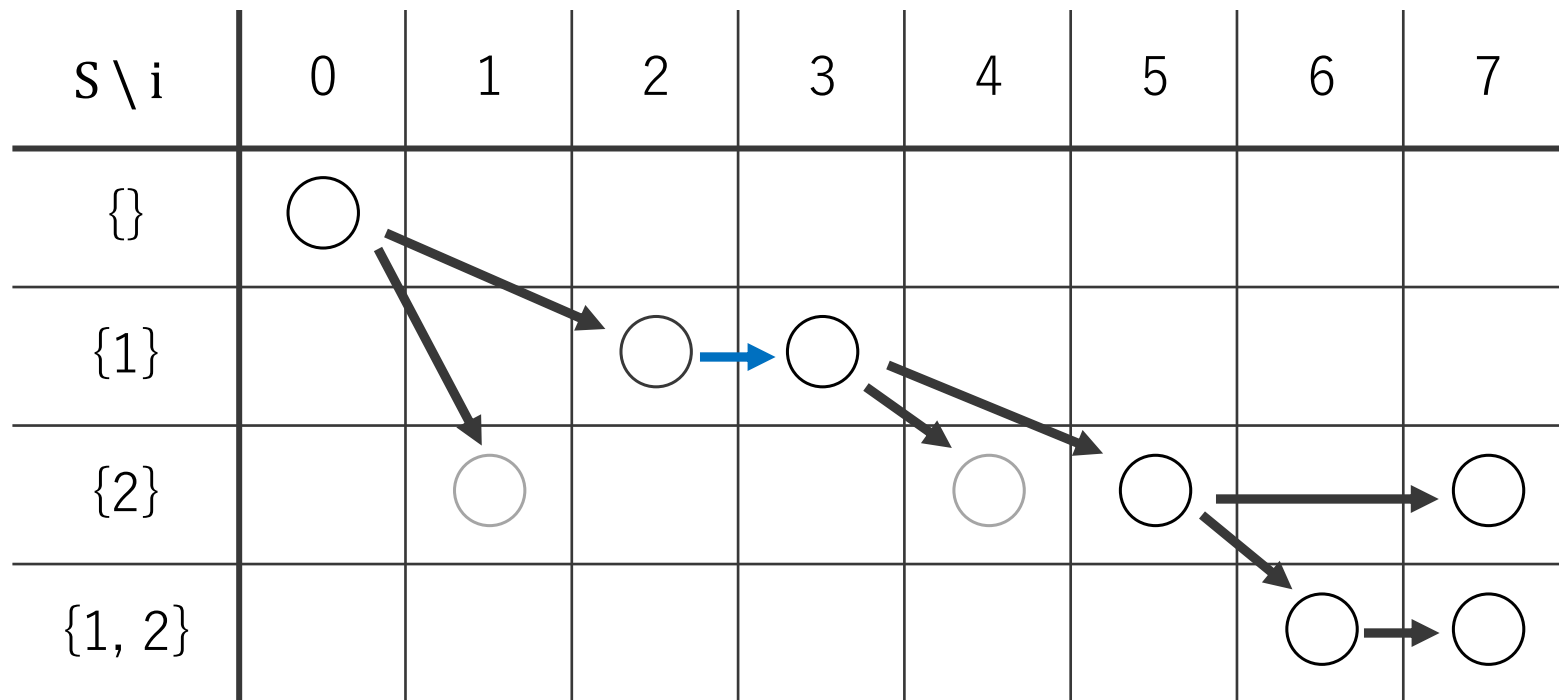
$$A = \begin{bmatrix} 2 & 1 & 1 & 2 & 2 & 1 & 2 \end{bmatrix}$$



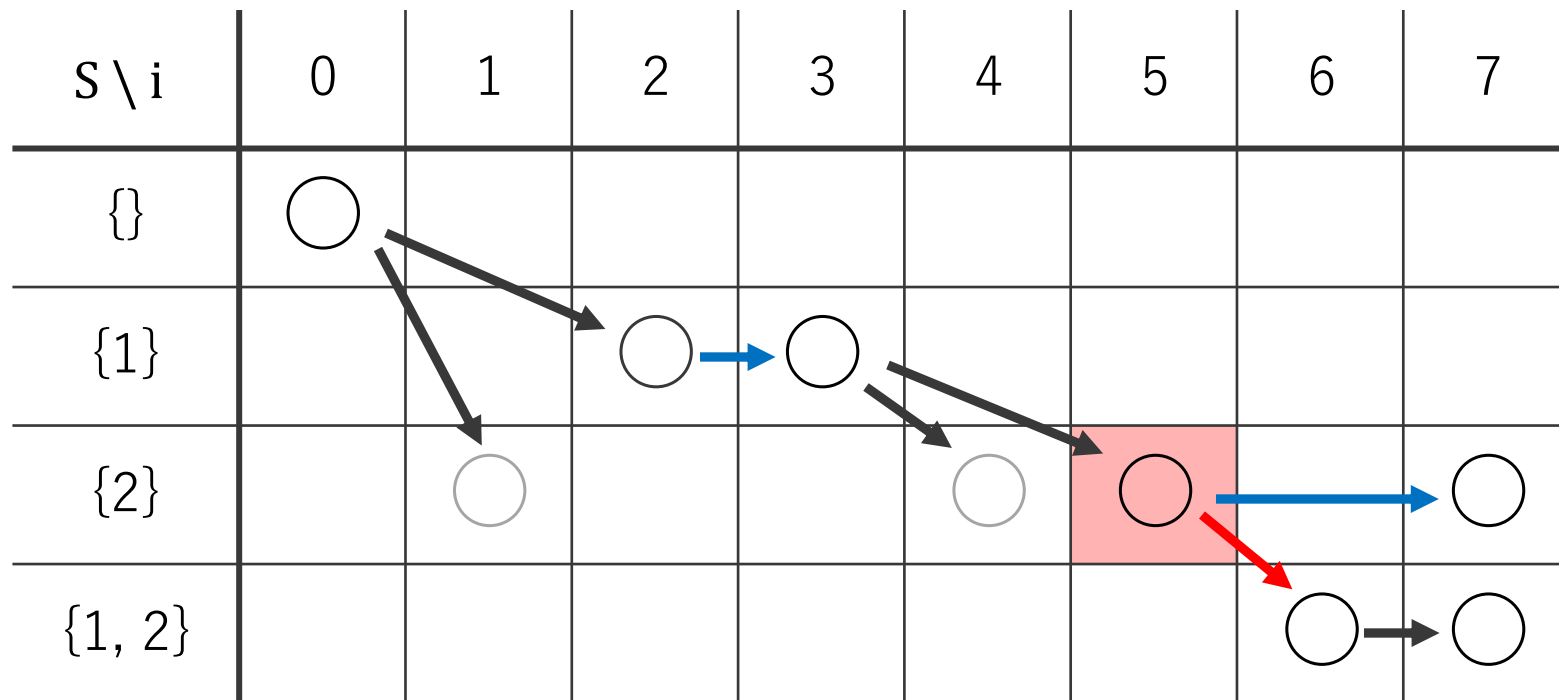
$$A = \begin{bmatrix} 2 & 1 & 1 & 2 & 2 & 1 & 2 \end{bmatrix}$$



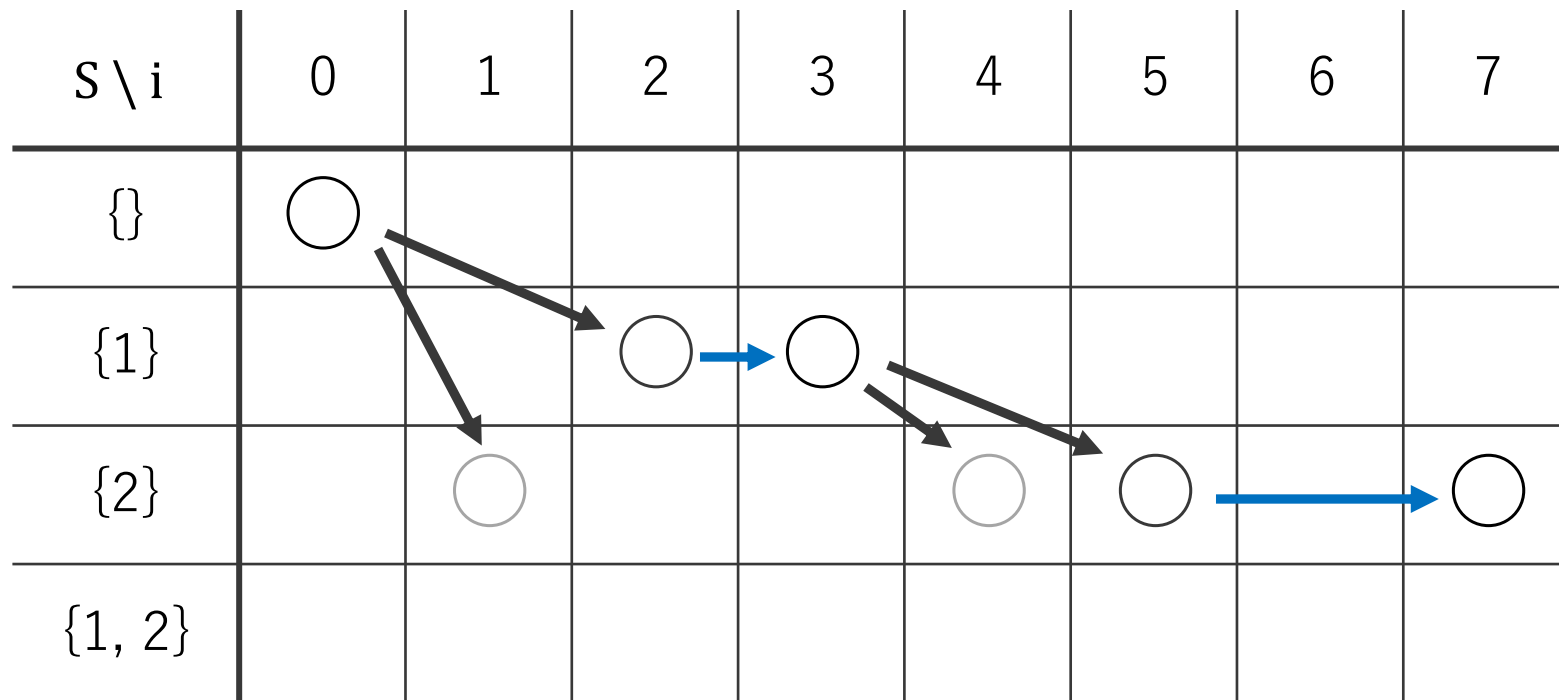
$$A = \begin{bmatrix} 2 & 1 & 1 & 2 & 2 & 1 & 2 \end{bmatrix}$$



$$A = \begin{bmatrix} 2 & 1 & 1 & 2 & 2 & 1 & 2 \end{bmatrix}$$



$$A = \begin{bmatrix} 2 & 1 & 1 & 2 & 2 & 1 & 2 \end{bmatrix}$$



考察 4: DP 高速化

- 小課題 4 で用いた bit DP の復習
 - $N2^L$ 個の bool 値を保持している
 - 保持する情報の量を減らしたい
 - 不要な情報 はないだろうか？
- 各 S について、 $dp_{i,S} = \text{true}$ となる最大の i さえ分かればよい
(各行について、最も右にある ○ の位置さえ分かればよい)

考察 4: DP 高速化

DP テーブルの各行 S について、

- $R(S)$: $\bigcirc$ が書き込まれる最も右の位置

これを求めるためには、以下の値が必要：

- $R_{in}(S)$: **他の行** からの矢印が伸びてきている最も右の位置

考察 4: DP 高速化

$dp_{i,S}$ から $dp_{j,T}$ への遷移があるとき、
常に $S \leq T$ が成立するため

S (の要素を昇順に並べた列) の辞書順に $R_{in}(S), R(S)$ を求められる

- $R_{in}(S)$: これまでに処理した行から行 S への遷移 (矢印) の中で最も右にある遷移先の位置
- $R(S)$: $R_{in}(S)$ からスタートして、行 S 内での遷移を繰り返して到達できる最も右の位置
- 行 S から他の行への遷移は、 $R(S)$ からのものだけでよい

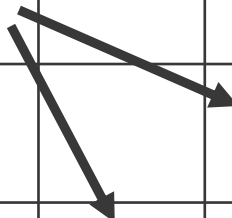
$$A = \begin{array}{|c|c|c|c|c|c|c|} \hline 2 & 1 & 1 & 2 & 2 & 1 & 2 \\ \hline \end{array}$$

$S \setminus i$	0	1	2	3	4	5	6	7
$\{\}$								
$\{1\}$								
$\{2\}$								
$\{1, 2\}$								

$A =$

2	1	1	2	2	1	2
---	---	---	---	---	---	---

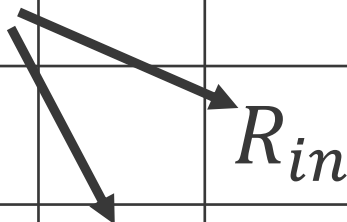
$S \setminus i$	0	1	2	3	4	5	6	7
$\{\}$								
$\{1\}$								
$\{2\}$								
$\{1, 2\}$								



The diagram shows two arrows originating from the circle in the row $\{\}$ at column 0. One arrow points to the circle in the row $\{1\}$ at column 2. The other arrow points to the circle in the row $\{2\}$ at column 1.

$$A = \begin{bmatrix} 2 & 1 & 1 & 2 & 2 & 1 & 2 \end{bmatrix}$$

$S \setminus i$	0	1	2	3	4	5	6	7
$\{\}$	○							
$\{1\}$								
$\{2\}$			○					
$\{1, 2\}$								



$$A = \begin{bmatrix} 2 & 1 & 1 & 2 & 2 & 1 & 2 \end{bmatrix}$$

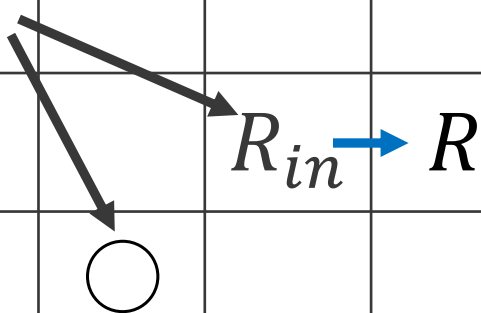
$S \setminus i$	0	1	2	3	4	5	6	7
$\{\}$	○							
$\{1\}$								
$\{2\}$								
$\{1, 2\}$								

Diagram illustrating a dynamic programming table for a subset problem. The table has rows representing subsets S and columns representing indices i . The initial state is at $(0, \{\})$. Transitions are shown by arrows:

- A black arrow points from $(0, \{\})$ to $(1, \{2\})$.
- A black arrow points from $(0, \{\})$ to $(3, \{1\})$.
- A blue arrow labeled R_{in} points from $(3, \{1\})$ to the right.

$$A = \begin{bmatrix} 2 & 1 & 1 & 2 & 2 & 1 & 2 \end{bmatrix}$$

$S \setminus i$	0	1	2	3	4	5	6	7
$\{\}$	○							
$\{1\}$								
$\{2\}$			○					
$\{1, 2\}$								



$$A = \begin{bmatrix} 2 & 1 & 1 & 2 & 2 & 1 & 2 \end{bmatrix}$$

$S \setminus i$	0	1	2	3	4	5	6	7
$\{\}$	○							
$\{1\}$								
$\{2\}$		○			○	○		
$\{1, 2\}$								

Diagram illustrating a sequence of set partitions or transitions:

- From $\{\}$ at $i=0$, transitions lead to $\{2\}$ at $i=1$ and $\{2\}$ at $i=2$.
- From $\{2\}$ at $i=2$, a transition labeled R leads to $\{2\}$ at $i=4$.
- From $\{2\}$ at $i=4$, a transition labeled R leads to $\{2\}$ at $i=5$.

$$A = \begin{bmatrix} 2 & 1 & 1 & 2 & 2 & 1 & 2 \end{bmatrix}$$

$S \setminus i$	0	1	2	3	4	5	6	7
$\{\}$	○							
$\{1\}$			R_{in}	$\rightarrow R$				
$\{2\}$		○			○	R_{in}		
$\{1, 2\}$								

$$A = \begin{bmatrix} 2 & 1 & 1 & 2 & 2 & 1 & 2 \end{bmatrix}$$

$S \setminus i$	0	1	2	3	4	5	6	7
$\{\}$	○							
$\{1\}$			R_{in}	R				
$\{2\}$		○			○	R_{in}		○
$\{1, 2\}$								

$$A = \begin{bmatrix} 2 & 1 & 1 & 2 & 2 & 1 & 2 \end{bmatrix}$$

$S \setminus i$	0	1	2	3	4	5	6	7
$\{\}$	○							
$\{1\}$			R_{in}	R				
$\{2\}$		○			○	R_{in}	R	
$\{1, 2\}$								

$$A = \begin{bmatrix} 2 & 1 & 1 & 2 & 2 & 1 & 2 \end{bmatrix}$$

$S \setminus i$	0	1	2	3	4	5	6	7
$\{\}$	○							
$\{1\}$			R_{in}	R				
$\{2\}$		○			○	R_{in}		R
$\{1, 2\}$								

考察 4: DP 高速化

計算量を考える

- 保持する情報は $R_{in}(S), R(S)$ のみなので $O(2^L)$ 個
- 行う計算は主に以下の二つ
 - 各 S について、 $R_{in}(S)$ から $R(S)$ を求める
 - 各 S について、 $R(S)$ からの遷移を処理する

考察 4: DP 高速化

計算量を考える

- 保持する情報は $R_{in}(S), R(S)$ のみなので $O(2^L)$ 個
- 行う計算は主に以下の二つ
 - 各 S について、 $R_{in}(S)$ から $R(S)$ を求める
 - 各 S について、 $R(S)$ からの遷移を処理する ← $O(2^L \cdot L)$

考察 4: DP 高速化

計算量を考える

- 保持する情報は $R_{in}(S), R(S)$ のみなので $O(2^L)$ 個
- 行う計算は主に以下の二つ
 - 各 S について、 $R_{in}(S)$ から $R(S)$ を求める ← これは？
 - 各 S について、 $R(S)$ からの遷移を処理する ← $O(2^L \cdot L)$

考察 4: DP 高速化

- $R_{in}(S)$ から同じ行内で右に進めるだけ進んだ位置が $R(S)$
- 「同じ行内で右に進める」とは？
 - S の値を変えない遷移が存在する
 - = 1 個先 / 2 個先の要素の少なくとも一方が集合 S に含まれる
- $R(S) : A[R_{in}(S)], A[R_{in}(S) + 1], \dots, A[N]$ の中で、
 S に含まれない要素が最初に連続して現れる場所

以下のクエリが $O(2^L)$ 回飛んでくるので、オンラインで処理せよ：

$Query(S, p) : A_p, A_{p+1}, \dots, A_N$ の中で、集合 S に含まれない要素が

連続して現れる最初の場所はどこか？

- あとはこの問題を解けばよい
- 様々な方針が存在し、速さに応じてどの小課題まで取れるか決まる

以下のクエリが $O(2^L)$ 回飛んでくるので、オンラインで処理せよ：

$Query(S, p) : A_p, A_{p+1}, \dots, A_N$ の中で、集合 S に含まれない要素が

連続して現れる最初の場所はどこか？

- 愚直に探す： $O(2^L N)$
 - 実測上ではかなり高速に動作する

以下のクエリが $O(2^L)$ 回飛んでくるので、オンラインで処理せよ：

$Query(S, p) : A_p, A_{p+1}, \dots, A_N$ の中で、集合 S に含まれない要素が
連続して現れる最初の場所はどこか？



以下のクエリが $O(2^L \cdot L^2)$ 回飛んでくるので、オンラインで処理せよ：

$Query(i, j, p) : A_p, A_{p+1}, \dots, A_N$ の中で、 i, j がこの順に連続して現れる
最初の場所はどこか？

以下のクエリが $O(2^L \cdot L^2)$ 回飛んでくるので、オンラインで処理せよ：

$Query(i, j, p) : A_p, A_{p+1}, \dots, A_N$ の中で、 i, j がこの順に連続して現れる

最初の場所はどこか？

- 二分探索： $O(N + 2^L L^2 \log N)$
 - 各 i, j について、 i, j が連続する場所を事前に列挙しておく

以下のクエリが $O(2^L \cdot L^2)$ 回飛んでくるので、オンラインで処理せよ：

$Query(i, j, p) : A_p, A_{p+1}, \dots, A_N$ の中で、 i, j がこの順に連続して現れる

最初の場所はどこか？

- 前計算： $O(NL^2 + 2^L L^2)$
 - クエリの入力は $O(NL^2)$ 通り存在する
 - i の降順にすべて前計算しておく

以下のクエリが $O(2^L \cdot L^2)$ 回飛んでくるので、オンラインで処理せよ：

$Query(i, j, p) : A_p, A_{p+1}, \dots, A_N$ の中で、 i, j がこの順に連続して現れる

最初の場合はどこか？

- バケット法： $O(N + 2^L L^2)$
 - p が L^2 の倍数であるような入力についてのみ前計算
 - そのような入力は $O(L \cdot L \cdot \frac{N}{L^2}) = O(N)$ 通り存在する
 - $Query(i, j, p)$ に答えるときは、 $L^2 \lceil \frac{p}{L^2} \rceil$ 番目までは愚直に見て、それ以降は前計算の値を用いる

まとめ

subtask 5 $N \leq 5 \times 10^5, A_i \leq 15$

subtask 6 $N \leq 5 \times 10^5, A_i \leq 21$

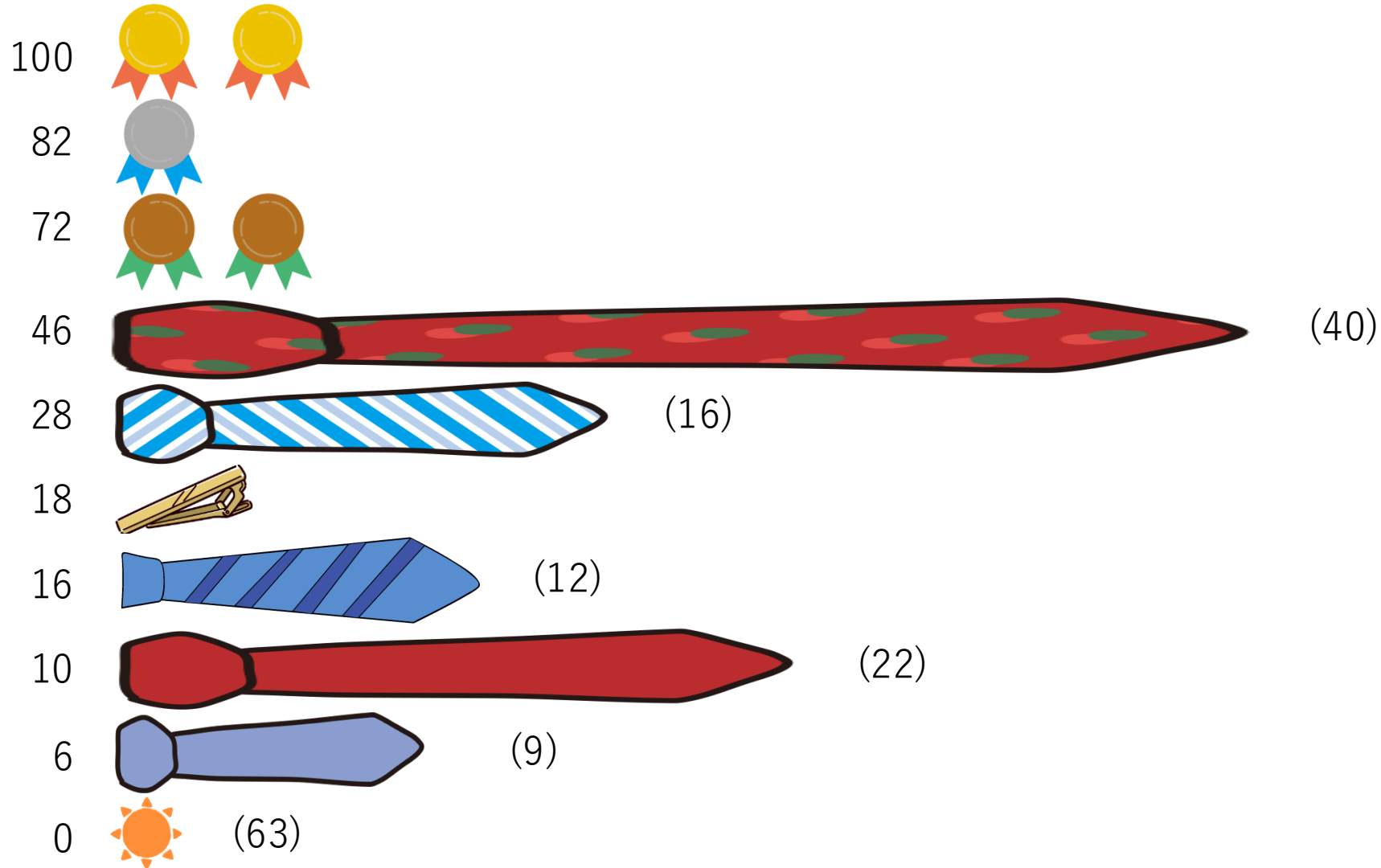
subtask 7 $N \leq 5 \times 10^6, A_i \leq 21$

	時間計算量	空間計算量	想定得点
愚直に探す	$O(2^L N)$	$O(N + 2^L)$	72 (~sub5)
二分探索	$O(N + 2^L L^2 \log N)$	$O(N + 2^L)$	72 (~sub5)
前計算	$O(NL^2 + 2^L L^2)$	$O(NL^2 + 2^L)$	82 (~sub6)
バケット法	$O(N + 2^L L^2)$	$O(N + 2^L)$	100 (~sub7)

別解

- Dijkstra 法の要領で、 $R_{in}(S)$ の昇順に $R_{in}(S), R(S)$ を求める
 - $Query(i, j, p)$ が p の昇順に飛んでくるので、
高速に処理することができ、満点を取れる
- Dilworth の定理を用いて考察することもできる
 - 結局同じ問題に帰着される

得点分布



5 - Post Office 解説

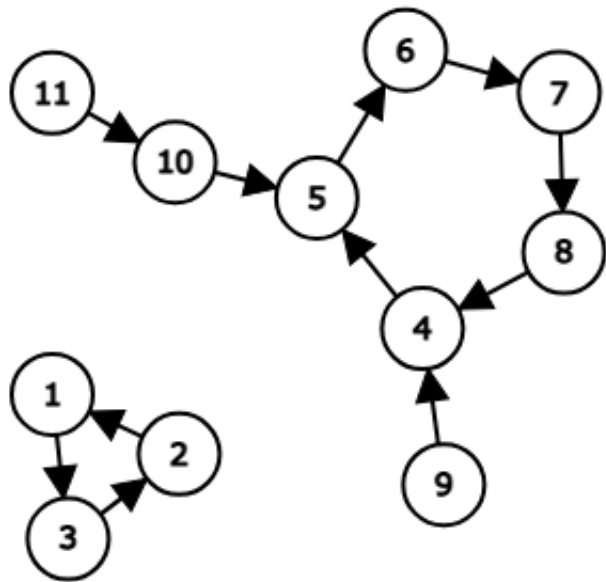
ynymxiaolongbao

問題概要

- N 頂点のFunctional Graph（有向なもり）が与えられる。
- M 個の荷物があり、頂点 $A[i]$ から頂点 $B[i]$ に、辺に沿って送っていく。
- 各時刻で、各頂点からは一つまで荷物を送れる。
- 最後に荷物の移動があった時刻の最小化（不可能ならそう主張）

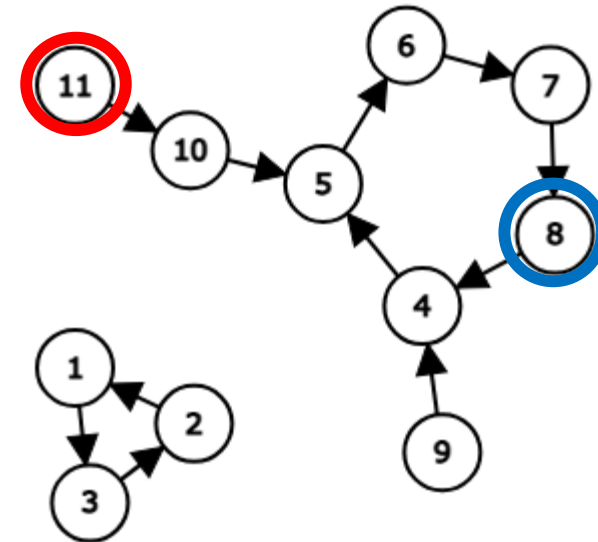
問題概要

- Functional Graph(有向なもり)
- なもりグラフの語源→
- sample 6 ↓



小課題 1 (3点)

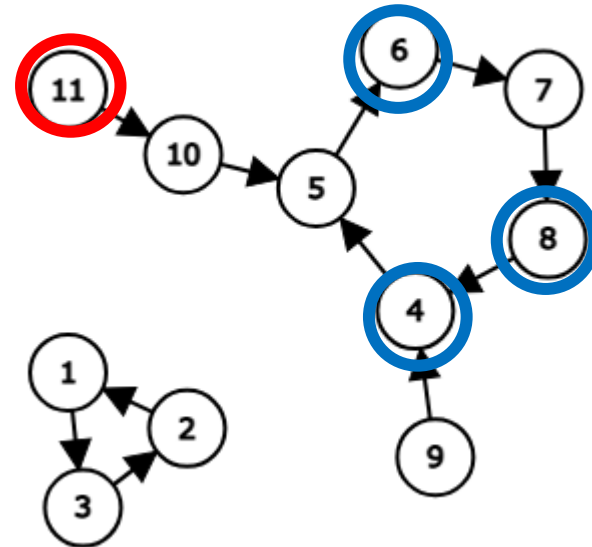
- $N = 3000, M = 1$
- s から t への距離を調べる
- 辺に沿って進んでいき、時刻 N までに到着しなかったら不可能
- $O(N)$



小課題 2 (9点)

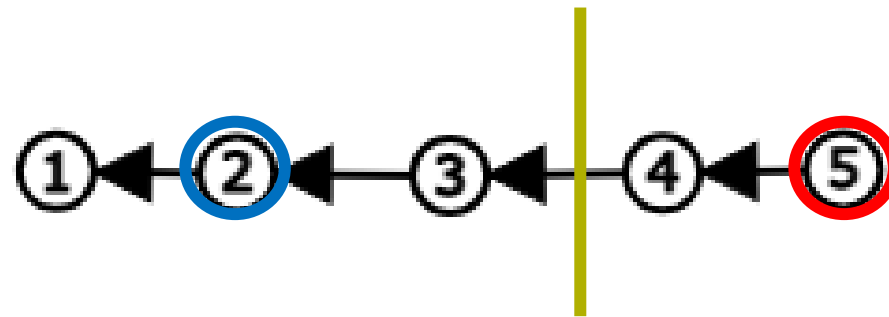
- $N = 3000, M = 3000$
- ある時刻、ある郵便局 に注目
- 荷物がいくつかあり、そのうちどれか一つを送る
- どれを送るか？ -> 最も送り先までの距離が遠いもの
- $O(NM)$

- 右の例では4,8,6の順に送る



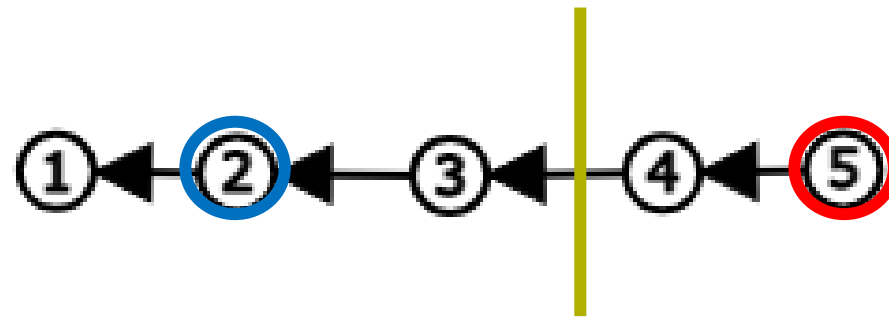
小課題 3 (13点)

- パス ある郵便局kをすべての荷物が通過する
- その郵便局kの気持ちになる
- 各荷物は以下の二つの情報で特徴付けられる
 - 郵便局kに到着する時刻
 - 郵便局kから目的地に到着するまでの時間



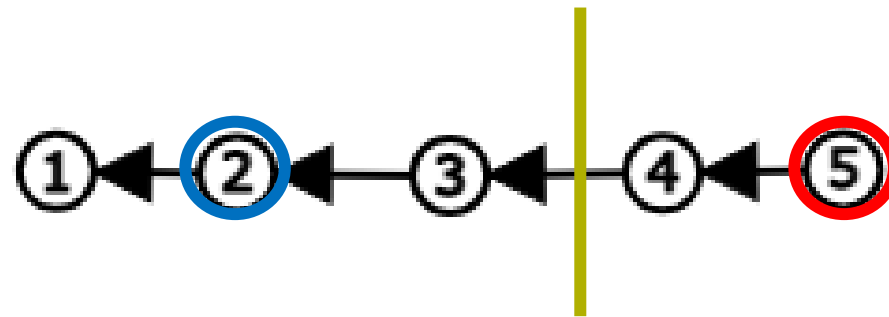
小課題 3 (13点)

- 郵便局kに到着する時刻 … すべての荷物が最短で到着する訳ではないが、最短で到着するとしてよい (最短で到着したとしても、結局郵便局kからは一つずつ発送するため)
- 郵便局kから目的地に到着するまでの時間 … 最短としてよい (郵便局kから一つずつ発送するため)



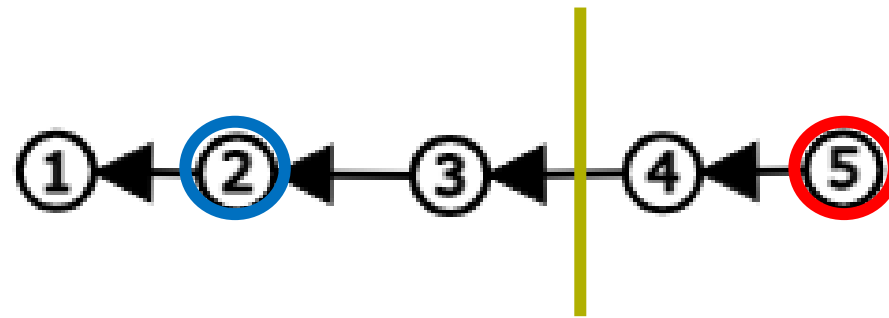
小課題 3 (13点)

- 各時刻で、郵便局kにある荷物のうち、目的地までの距離が最大のものを発送する
- priority queue で $O(N + M \log M)$



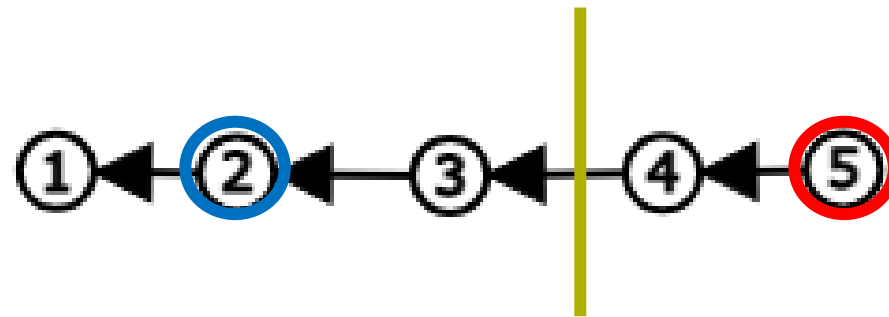
小課題 4 (25点)

- ある郵便局kに注目 ここから最後に発送する時刻を知りたい
- これに必要な情報は、各荷物について以下のみ
 - 郵便局kに到着する時刻
- 小課題 3 の考察より、最短で郵便局kに到着するとしてよい



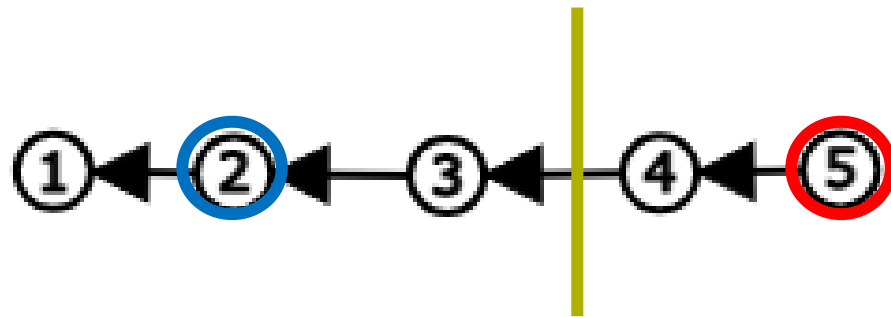
小課題 4 (25点)

- 郵便局 k から最後に発送する時刻は？
- 郵便局 k に最後に荷物が到着するまでのすべての時刻 t について、郵便局 k から最後に荷物を発送する時刻は、 t 以降に到着する荷物の個数 $+ t - 1$ 以上
- 逆にこれをみたせばよい



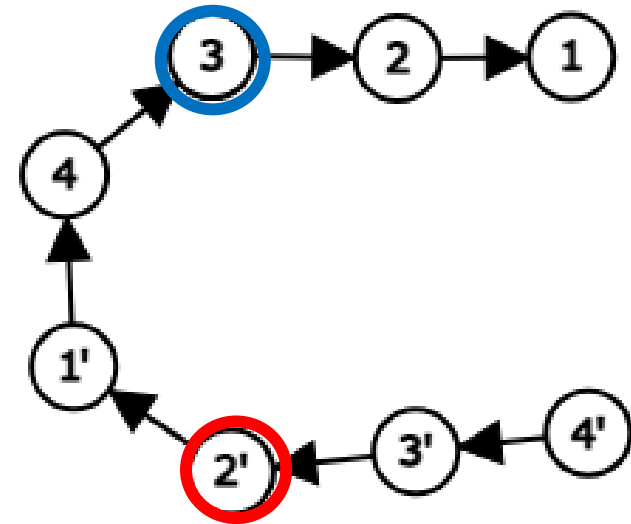
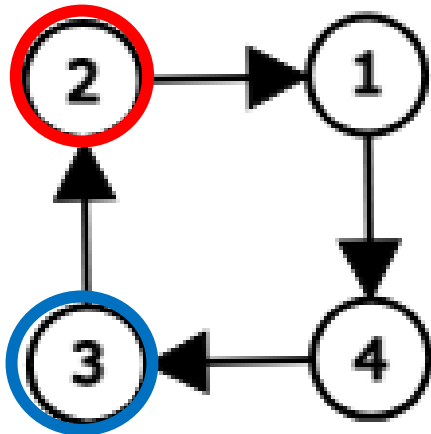
小課題 4 (25点)

- 区間加算区間maxのlazy segtree で管理しながらパスを辿る
- $O(N + M \log N)$



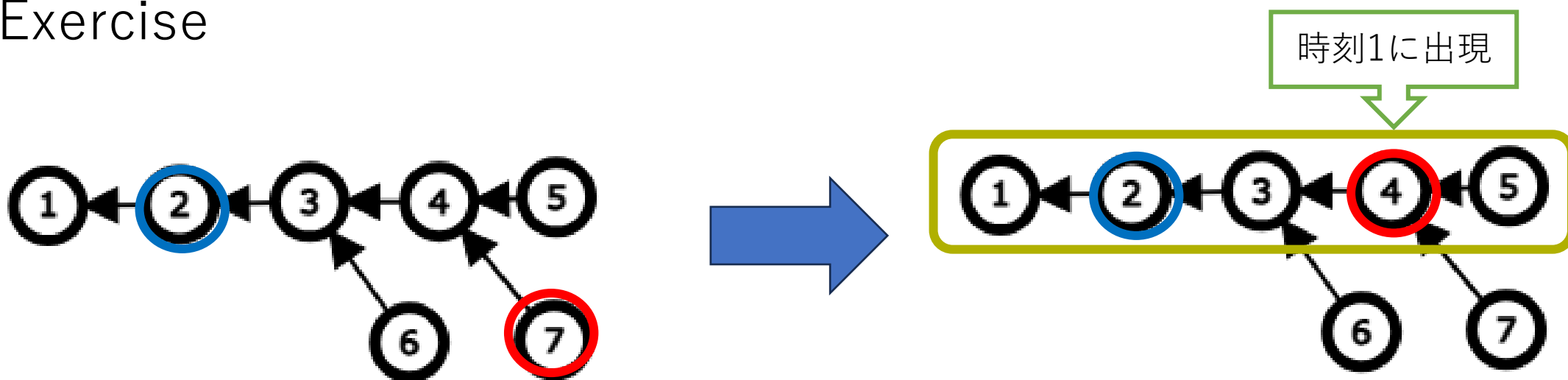
小課題 5 (11点)

- 円環
- 切り開いて長さ $2N$ のパスにすればOK
- Excercise



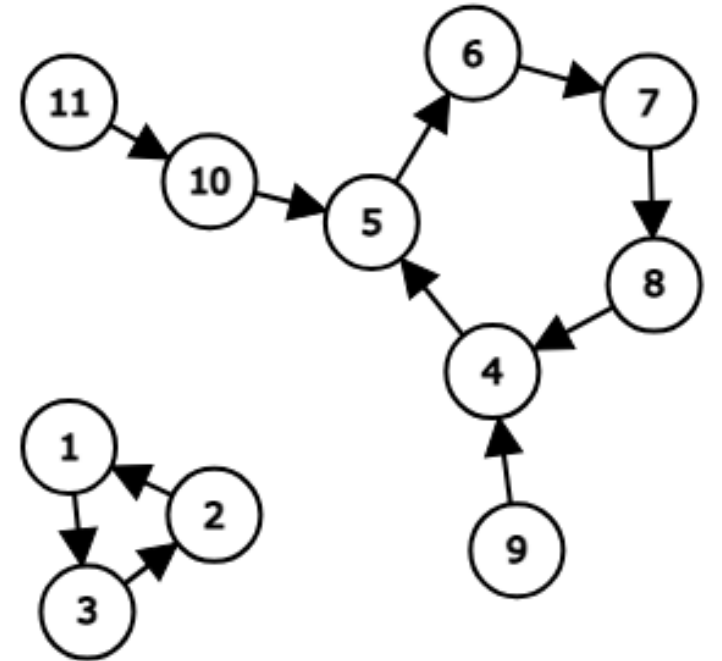
小課題 6 (25点)

- 木
- HLD(Heavy Light Decomposition)すると、パスの問題をいくつか解けば良くなる (荷物の個数の合計は $O(M \log N)$ になる)
- Exercise



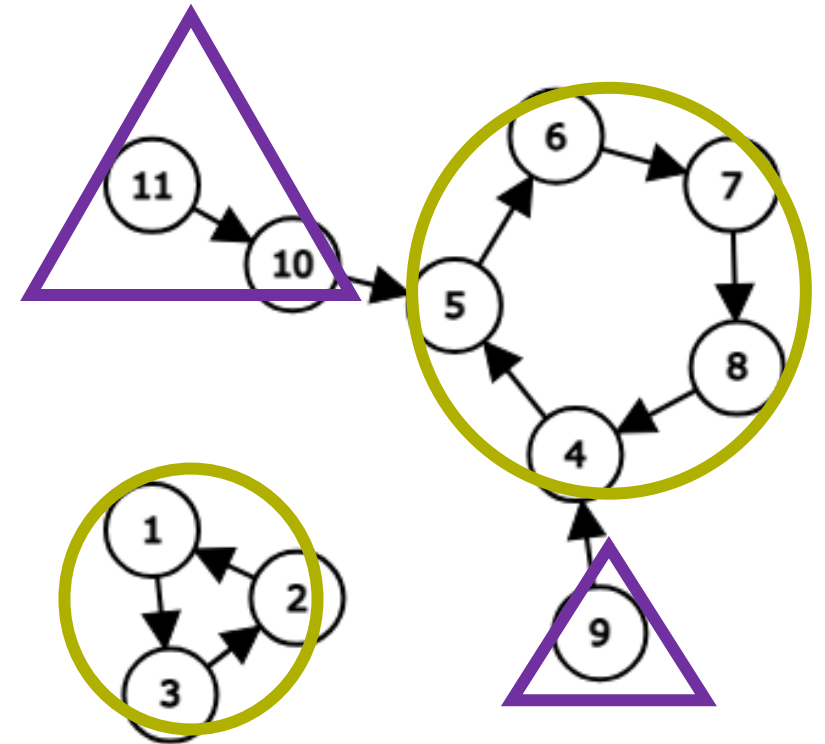
小課題 7 (14点)

- Functional Graph (有向なもり)
- 二つの見方
- 円環 + 木の列 (円環に根付き木がいくつか生えている)
- 木 + 一辺 (根付き木の根から一本辺が出ている)
- どちらでも、これまでの小課題にある二つに分けて解ける



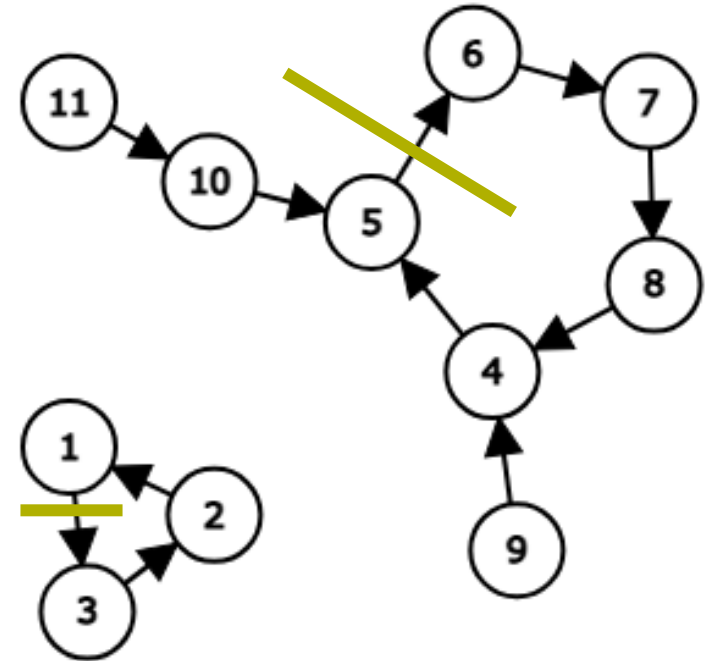
小課題 7 (14点)

- Functional Graph (有向なもり)
- 二つの見方
- **円環 + 木の列** (円環に根付き木がいくつ
か生えている)
- 木 + 一辺 (根付き木の根から一本辺が出
ている)
- どちらでも、これまでの小課題にある二
つに分けて解ける



小課題 7 (14点)

- Functional Graph (有向なもり)
- 二つの見方
- 円環 + 木の列 (円環に根付き木がいくつか生えている)
- **木 + 一辺** (根付き木の根から一本辺が出ている)
- どちらでも、これまでの小課題にある二つに分けて解ける



得点分布

- 50pts: 3
- 25pts: 2
- 13pts: 1
- 12pts: 15
- 3pts: 57
- 0pts: 90